

The background image features a golden, metallic robot head sculpture in the foreground, shown in profile. The robot's head is highly detailed, revealing internal mechanical components like gears and pistons. Behind the robot, a modern, multi-story building with a grid-like facade of windows and balconies is visible. The scene is set in an urban environment with a street and some blurred figures of people in the distance. The overall color palette is warm, dominated by the gold of the robot and the light tones of the building and sky.

CV项目

传统计算机视觉:OpenCV

课程内容

- 🕒 OpenCV概述
- 🕒 图像数据处理
- 🕒 图像特征及描述
- 🕒 图像分类
- 🕒 图像检测

OpenCV概述

⌚ OpenCV是一个开源的图像处理框架，主要提供了C++、Python、Java等版本的API，支持Windows、Linux、Mac OS、IOS以及Android等常规系统，是传统计算机视觉中最常用的一种图像视觉处理库。现阶段来讲，OpenCV对于深度深度学习来讲，主要的应用有两个：图像预处理、模型部署(PyTorch模型训练、可以用OpenCV进行部署)

⌚ 官网：<https://opencv.org/>

⌚ 源码：<https://github.com/opencv/opencv>

⌚ 文档：<https://docs.opencv.org/>



OpenCV安装

⌚ OpenCV的安装相对比较简单，可以直接从官网上找到安装方式，或者从pypi的官网找到安装方式：

⌚ 安装参考：https://docs.opencv.org/4.5.5/d0/d3d/tutorial_general_install.html

⌚ pypi安装方式：<https://pypi.org/project/opencv-python/4.5.5.62/>

⌚ whl文件安装：<https://www.lfd.uci.edu/~gohlke/pythonlibs/#opencv>

⌚ 安装命令：

⌚ `pip install opencv-python==4.5.5.64`

```
(base) PS C:\Users\19410> pip install opencv-python==4.5.5.62
Defaulting to user installation because normal site-packages is not writeable
WARNING: Ignoring invalid distribution -rotobuf (c:\users\19410\appdata\roaming\python\python39\site-packages)
WARNING: Ignoring invalid distribution -rotobuf (c:\users\19410\appdata\roaming\python\python39\site-packages)
Looking in indexes: https://mirrors.aliyun.com/pypi/simple
Collecting opencv-python==4.5.5.62
  Downloading https://mirrors.aliyun.com/pypi/packages/65/d4/0f4b0f72687780e3b4662e4a5323736d6c534e6914508e6239914c8d5/opencv_python-4.5.5.62-cp36-abi3-win_amd64.whl (35.4 MB)
    35.4/35.4 MB 2.5 MB/s eta 0:00:00
Requirement already satisfied: numpy>=1.17.3 in c:\users\19410\appdata\roaming\python\python39\site-packages (from opencv-python==4.5.5.62) (1.19.5)
WARNING: Ignoring invalid distribution -rotobuf (c:\users\19410\appdata\roaming\python\python39\site-packages)
Installing collected packages: opencv-python
WARNING: Ignoring invalid distribution -rotobuf (c:\users\19410\appdata\roaming\python\python39\site-packages)
Successfully installed opencv-python-4.5.5.62
WARNING: Ignoring invalid distribution -rotobuf (c:\users\19410\appdata\roaming\python\python39\site-packages)
WARNING: Ignoring invalid distribution -rotobuf (c:\users\19410\appdata\roaming\python\python39\site-packages)
WARNING: Ignoring invalid distribution -rotobuf (c:\users\19410\appdata\roaming\python\python39\site-packages)
(base) PS C:\Users\19410> python
Python 3.9.13 (main, Aug 25 2022, 23:51:50) [MSC v.1916 64 bit (AMD64)] :: Anaconda, Inc. on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import cv2
>>> cv2.__version__
'4.5.5'
>>>
```

图像存储原理

⌚ RGB颜色空间

⌚ 加法混色，彩色显示器

⌚ 3通道：

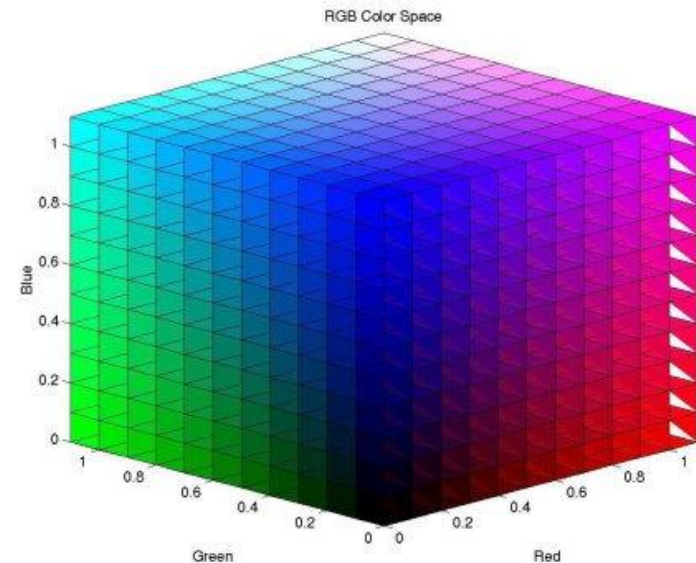
⌚ Red通道

⌚ Green通道

⌚ Blue通道

⌚ 取值范围：[0,255]、[0.0,1.0]

⌚ 特殊颜色像素值：白色(255,255,255)、黑色(0,0,0)



图像存储原理

⌚ CMY(K)颜色空间

⌚ 减法混色，印刷颜色

⌚ 4通道：

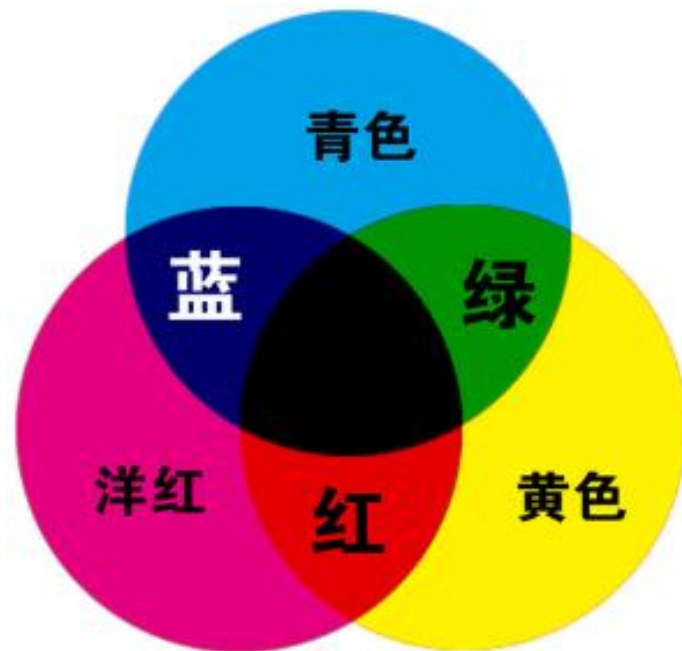
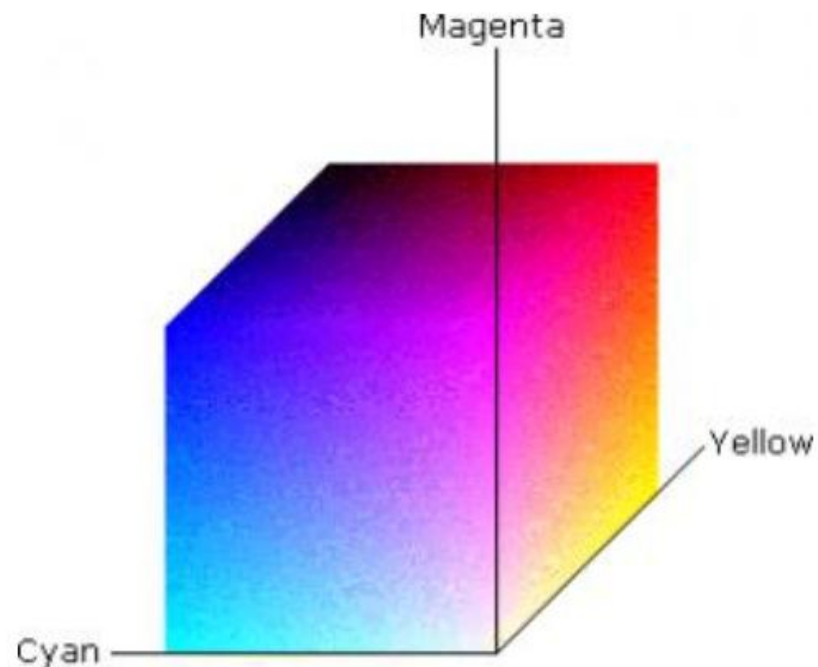
⌚ Cyan通道

⌚ Magenta通道

⌚ Yellow通道

⌚ Key通道

⌚ 取值范围：[0,255]、[0.0,1.0]



图像存储原理

⌚ HSV颜色空间

⌚ 人类视觉概念

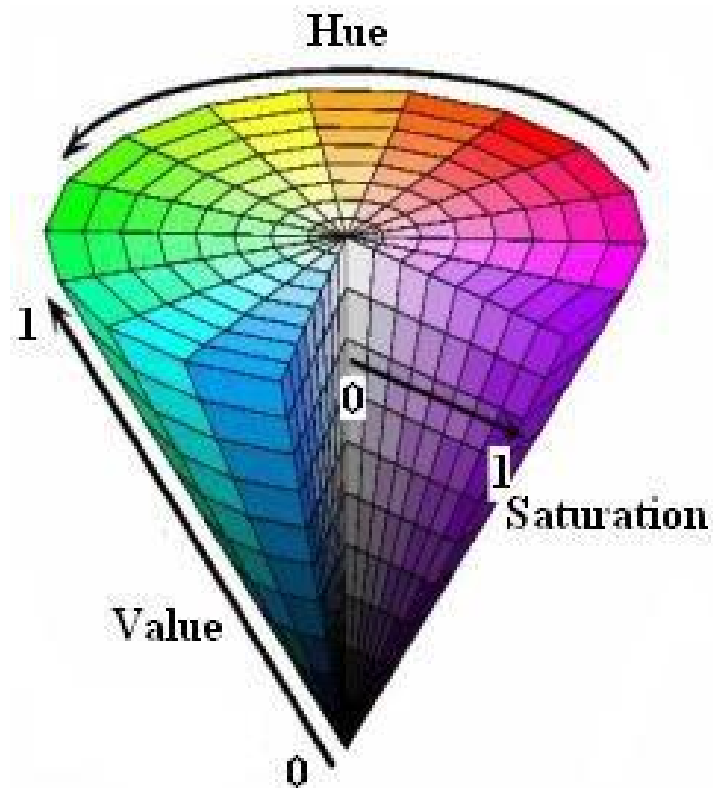
⌚ 3通道

⌚ Hue: 色调, 颜色种类

⌚ Saturation: 饱和度, 颜色的浓淡

⌚ Value: 明度, 颜色明亮度

⌚ 在不同平台上取值不是特别一致!



图像存储原理

⌚ 主流颜色空间

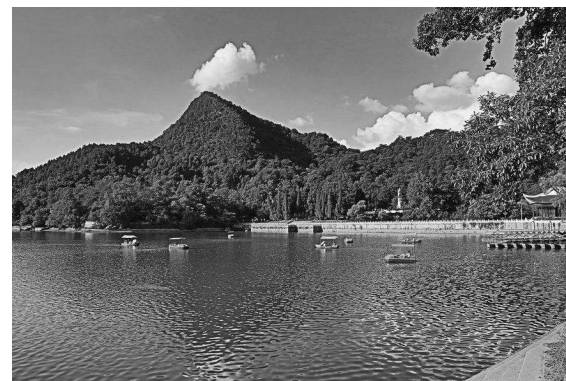
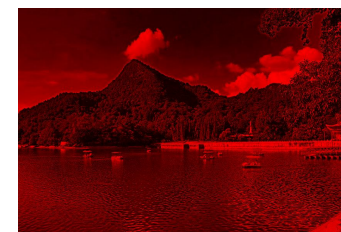
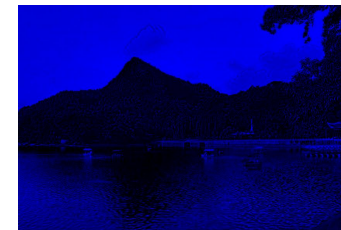
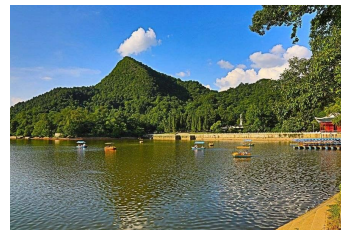
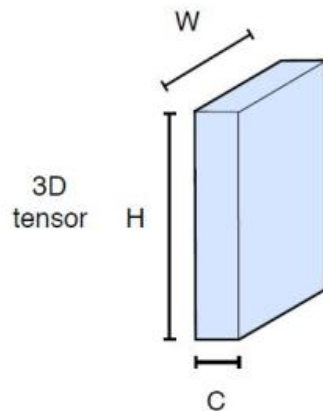
⌚ RGB三通道彩色图

⌚ 图片 -> 3维矩阵([0,255])

⌚ 单通道灰度图

⌚ 亮度信息([0,255])

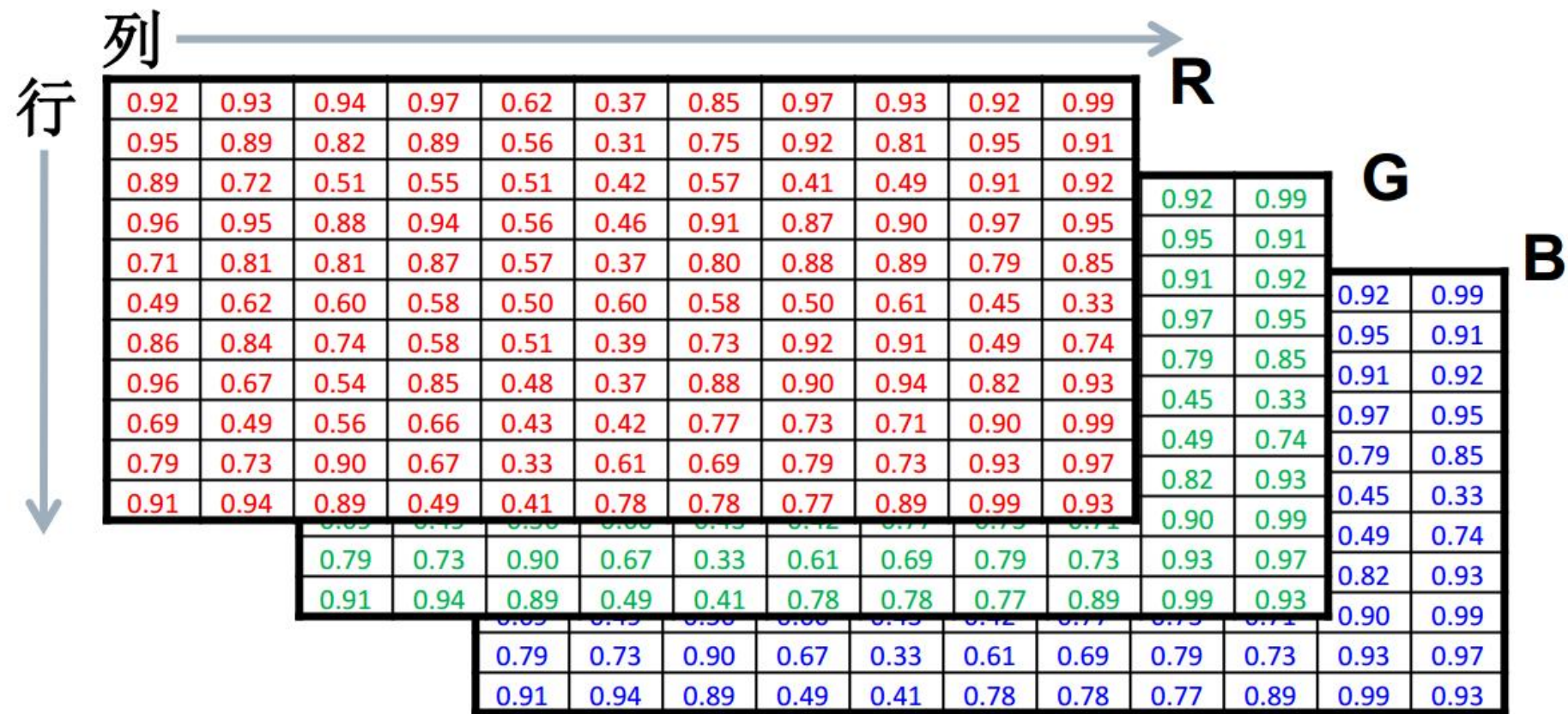
⌚ 简单计算方式: $\text{Gray} = R \cdot 0.3 + G \cdot 0.59 + B \cdot 0.11$



图片存储原理

🕒 RGB图片数据格式

🕒 3维矩阵



OpenCV基本操作

- ⌚ 图像的基本处理（读、写、可视化）
- ⌚ 视频的基本处理（播放视频、捕获视频以及保存视频）
- ⌚ 基于OpenCV的绘图（绘制直线、矩形、圆等内容）

OpenCV基本操作

API	描述
cv.imread	根据给定的磁盘路径加载对应的图像，默认使用BGR方式加载。
cv.imshow	展示图像
cv.imwrite	将图像保存到磁盘中
cv.waitKey	暂停一段时间，接受键盘输出后，继续执行程序
cv.destroyAllWindows	释放所有资源
cv.VideoCapture	获取摄像头对象并打开，或者获取某一个视频文件内容
cv.line	在图像中绘制直线
cv.circle	在图像中绘制圆
cv.rectangle	在图像中绘制矩形
cv.ellipse	在图像中绘制椭圆
cv.polylines	在图像中绘制多边形
cv.putText	在图像中绘制文字

OpenCV算术运算

⌚ 像素操作

⌚ NOTE: 可以直接将OpenCV读取的图像当作NumPy的数组来进行操作。

⌚ 图像分割及合并

⌚ 边框操作

⌚ NOTE: 也就是在原始图像的周边添加一些新的像素。

⌚ 图像位运算

OpenCV算术运算

API	描述
img.item	获取对应行、对应列的对应通道的像素值
img.itemset	设置对应行、对应列的对应通道的像素值
cv.split	将图像划分为对应的通道
cv.merge	将图像的各个通道进行合并，产生一个新的图像
cv.copyMakeBorder	在图像中加入边框
cv.addWeighted	将两个图像合并到一起，直接重叠合并到一起
cv.bitwise_not	图像位运算，对图像的每个像素值进行“非”操作，即： <code>dst=np.uint8(~src)</code>
cv.bitwise_and	图像位运算，对两个图像的每个像素值之间进行“与”操作，即： <code>dst=np.uint8(src1 & src2)</code>
cv.bitwise_or	图像位运算，对两个图像的每个像素值之间进行“或”操作，即： <code>dst=np.uint8(src1 src2)</code>
cv.bitwise_xor	图像位运算，对两个图像的每个像素值之间进行“异或”操作，即： <code>dst=np.uint8(src1 ^ src2)</code>

OpenCV图像几何变换

⌚ 颜色空间转换

⌚ 在OpenCV中一般常用的颜色空间为BGR，可以将其转换为GARY；其它颜色空间一般不做太多要求。

⌚ 图像大小缩放

⌚ 图像平移

⌚ 水平 or 垂直平移图像，对于没有原图像的位置直接填充黑色。

OpenCV图像几何变换

⌚ 图像旋转

⌚ 仿射变换

⌚ 仿射变换不会改变原始图像中点的特征信息，比如：点之间的共线性

⌚ 透视变换

⌚ 透视变换就是利用透视中心、像点以及目标点三点共线的条件进行旋转，最终透视变换之后的结果会保证几何图形不变性。

⌚ 二值化图像

⌚ 将灰度对象转换为黑白图像

OpenCV图像几何变换

API	描述
cv.cvtColor	对图像进行颜色空间转换
cv.resize	对图像进行大小重置操作
cv.warpAffine	使用给定的矩阵参数 M 对图像做一个平移、旋转、仿射变换等操作
cv.warpPerspective	使用给定的矩阵参数 M 对图像做一个透视变换
cv.threshold	对灰度图像做一个二值化操作
cv.adaptiveThreshold	对灰度图像做一个自适应的二值化操作

空域变换

⌚ 滤波/卷积

⌚ 在每个图像位置(x,y)上进行基于**邻域**的**函数**计算，其中函数参数被称为卷积核(kernel)，kernel核的尺寸大小一般为**奇数**，一般为:3*3、5*5、7*7居多，一般的函数操作位权重相加的操作。

⌚ 不同功能需要定义不同的函数

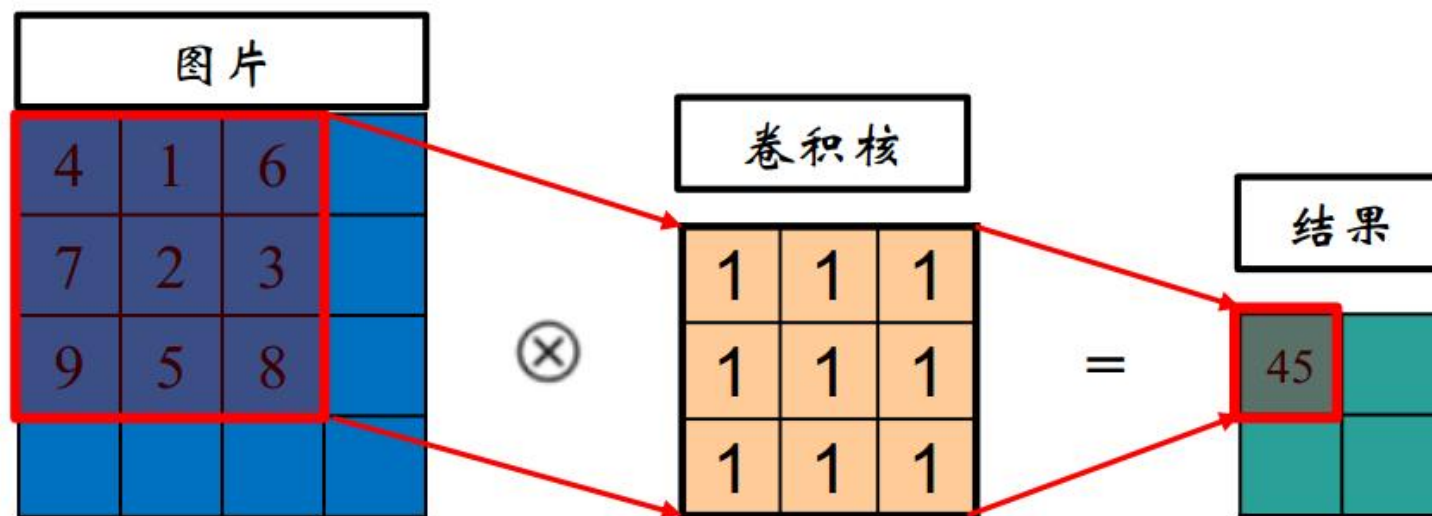
⌚ 图像增强

⌚ 平滑、去噪

⌚ 梯度、锐化

⌚ 特征提取、检测

⌚ 边缘、显著点



空域变换

⌚ 卷积:

⌚ 边界填充(Padding):

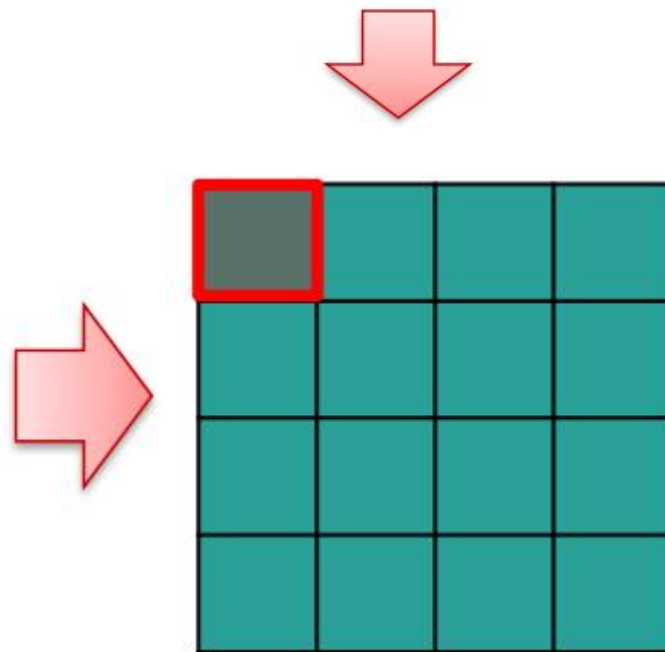
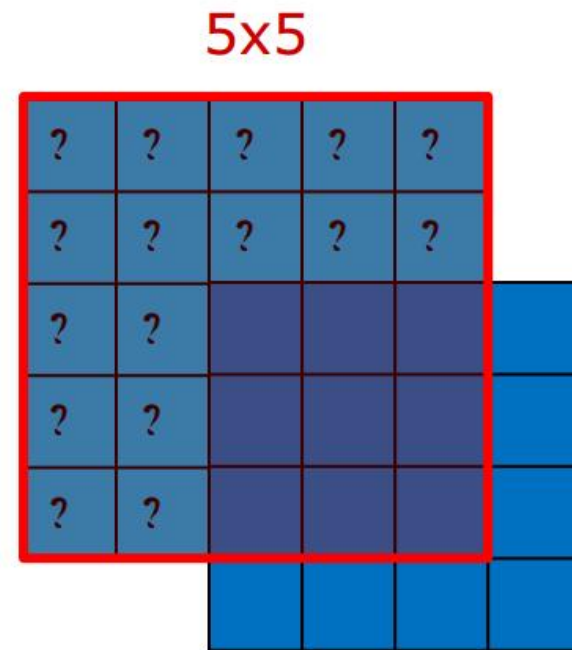
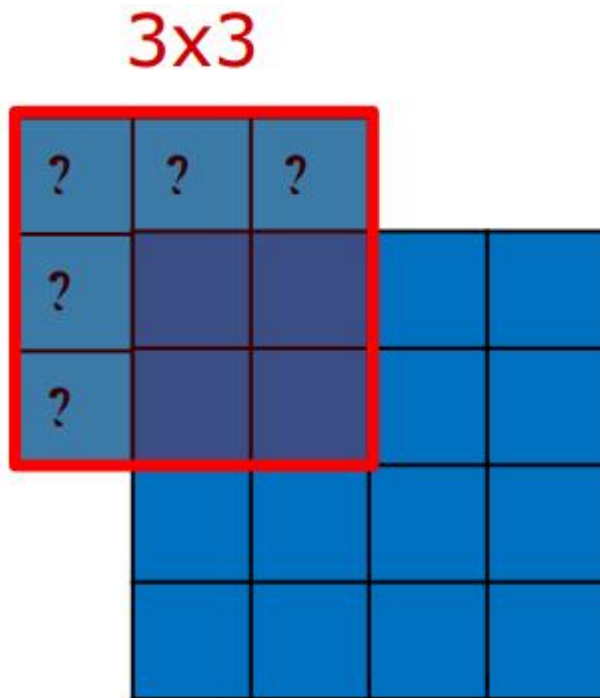
⌚ 在希望获取同尺寸输出的情况下, 需要进行必要的填充。

⌚ 填充API:

⌚ `cv.copyMakeBorder`

⌚ 常见填充类型:

- ⌚ 补常数(BORDER_CONSTANT)
- ⌚ 边界复制(BORDER_REPLICATE)
- ⌚ 块复制(BORDER_WRAP)
- ⌚ 镜像(BORDER_REFLECT)

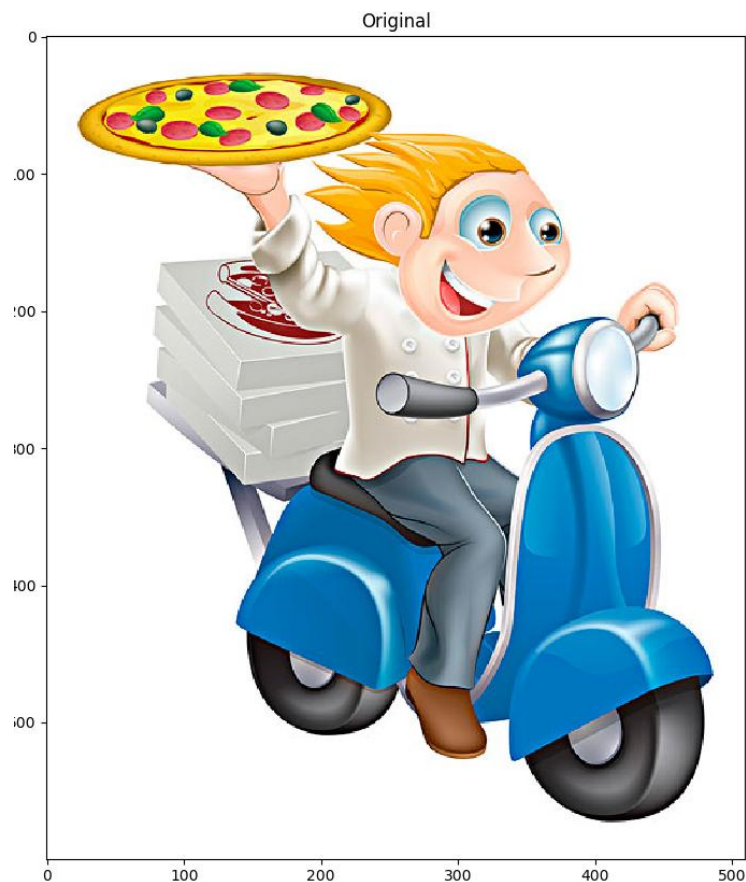


空域变换

⌚ 平滑**均值滤波**/卷积

⌚ kernel核中的参数和为1，所有参数值全部一样。

$$K = \frac{1}{25} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$



空域变换

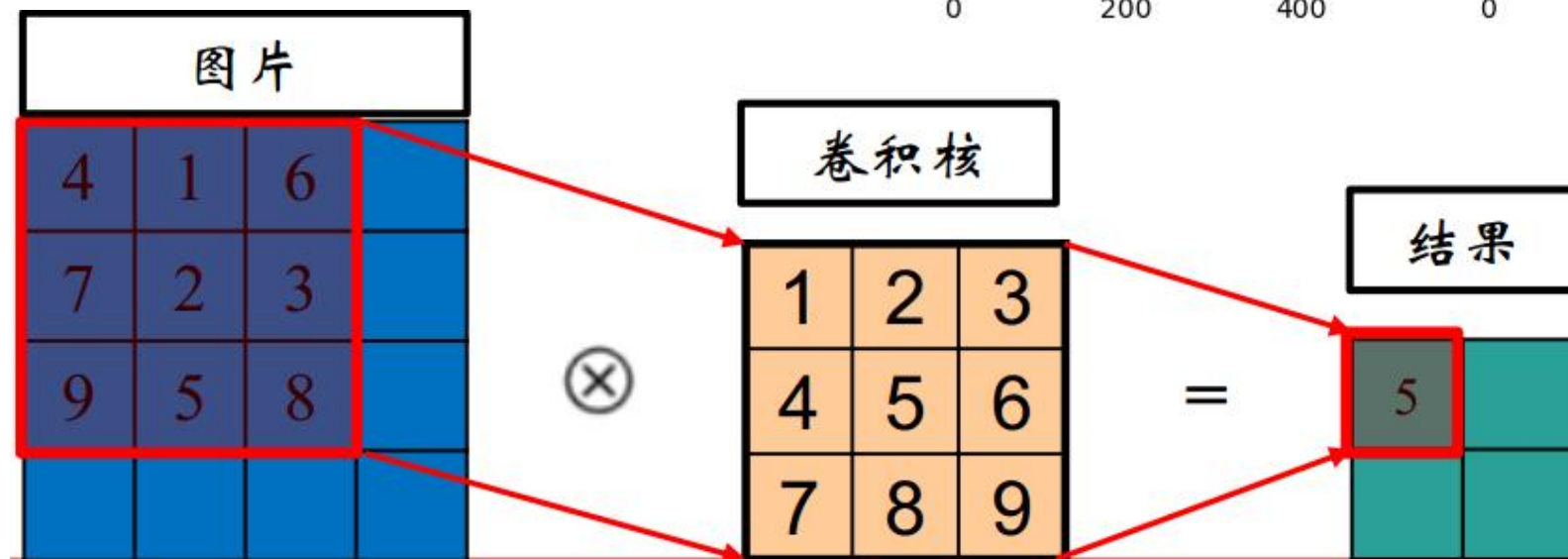
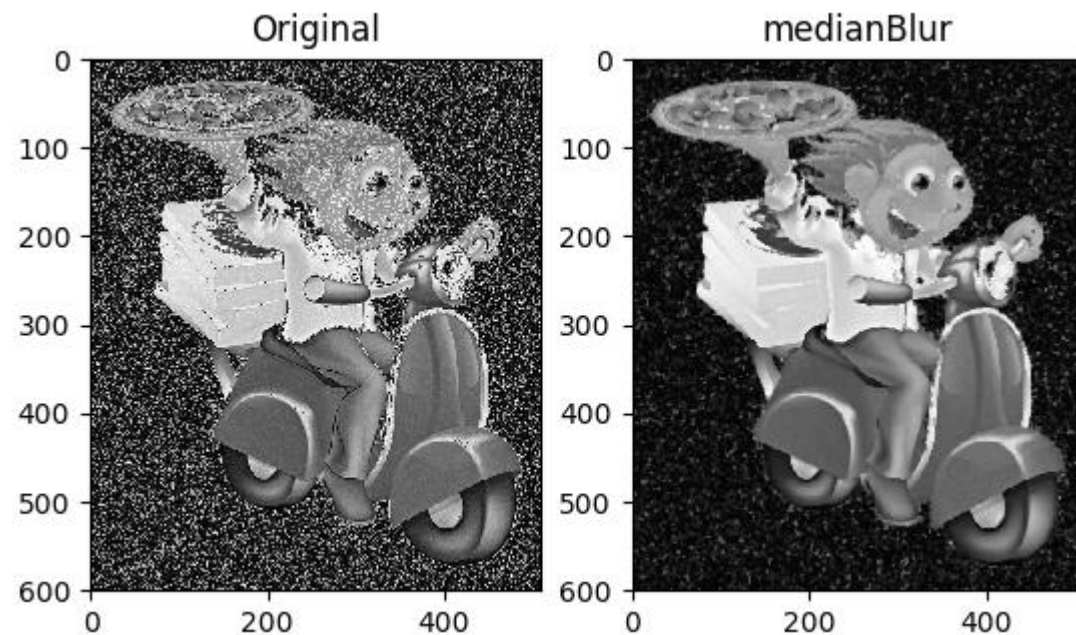
⌚ 平滑**中值滤波**/卷积

⌚ 操作原理:

⌚ 将卷积域内的所有像素值**从小到大排序**

⌚ 获取**中间值**作为卷积的输出值

⌚ 效果作用: 有效去除**椒盐**噪声



空域变换

⌚ 平滑**高斯滤波**/卷积

⌚ 模拟人眼，关注中心区域

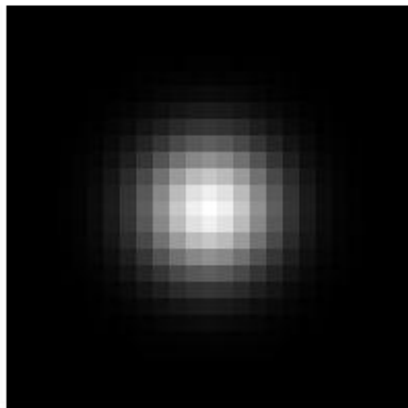
⌚ 可以有效去除**高斯噪声**

⌚ 参数：

⌚ x 、 y 是卷积参数坐标

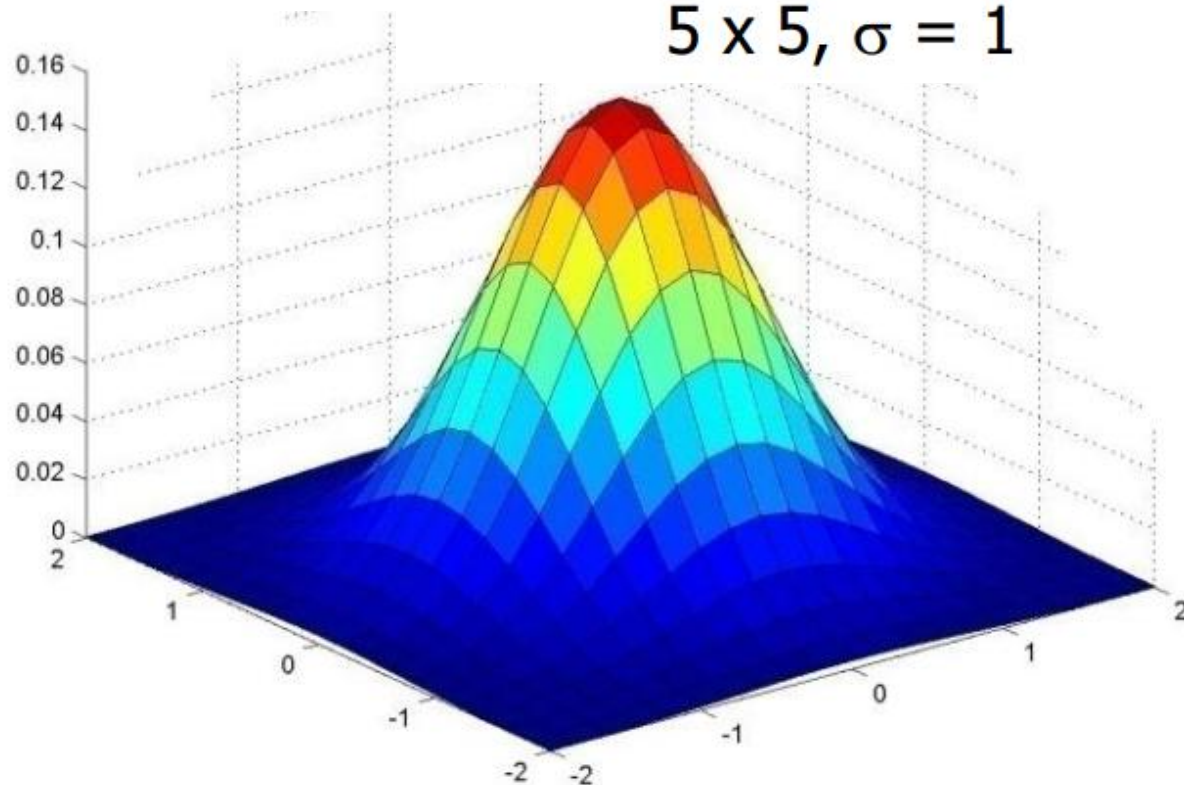
⌚ 标准差 σ

$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$



0.003	0.013	0.022	0.013	0.003
0.013	0.059	0.097	0.059	0.013
0.022	0.097	0.159	0.097	0.022
0.013	0.059	0.097	0.059	0.013
0.003	0.013	0.022	0.013	0.003

$5 \times 5, \sigma = 1$

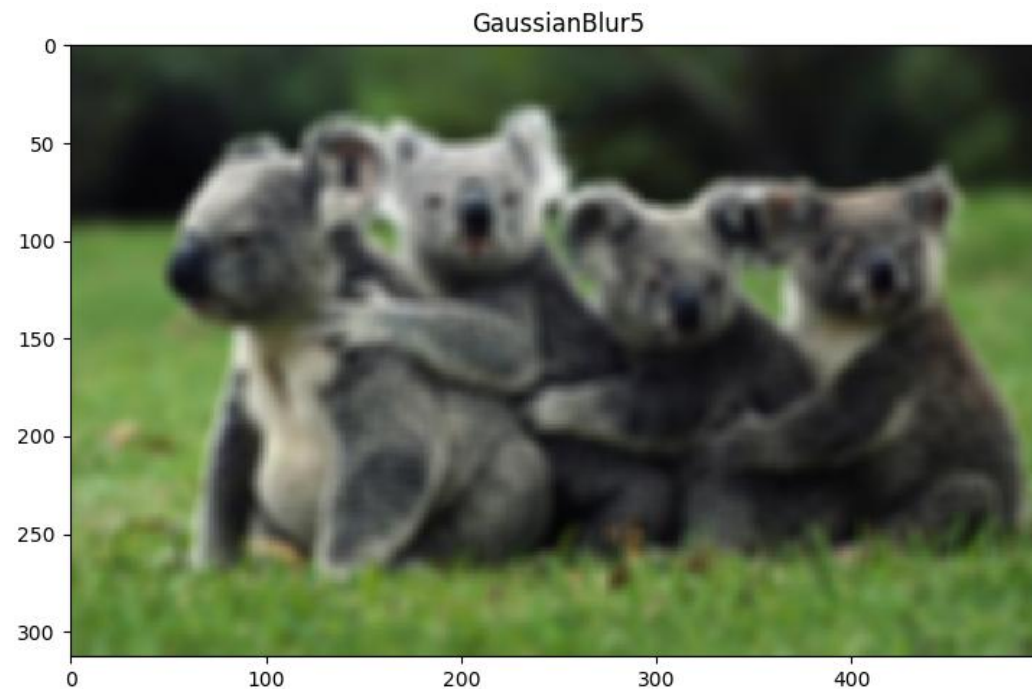


空域变换

⌚ 平滑高斯滤波/卷积

⌚ σ 越小，关注区域约小；

⌚ σ 越大，图片整体约模糊。



空域变换

⌚ 平滑高斯滤波/卷积

⌚ 分解特性（级联高斯）

⌚ 2D卷积拆分为两个相同的1D卷积

⌚ 列卷积

⌚ 行卷积

⌚ 降低计算

⌚ 2D卷积：K^2次计算

⌚ 1D卷积：2K次计算

1	2	1
2	4	2
1	2	1

 =

1
2
1

 X

1	2	1
---	---	---

$$G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$
$$= \left(\frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{x^2}{2\sigma^2}\right) \right) \left(\frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{y^2}{2\sigma^2}\right) \right)$$

空域变换

- ✂ 除了进行平滑模糊外，在卷积的操作中，还可以实现图片形态学变换的操作，主要包括**腐蚀(Erode)**、**膨胀(Dilate)**、**Open**、**Close**、**Morphological Gradient**、**Top Hat**、**Black Hat**等操作。
- ✂ 腐蚀：将图像的线条边缘像素去除，可以达到一定的去噪的效果；
- ✂ 膨胀：和腐蚀相反，可以增加图像中的白色区域的值，一般用于在腐蚀之后恢复增强目标区域的信息；
- ✂ Open：先做一次腐蚀、再做一次膨胀操作，一般用于去除**白色噪声**的处理；
- ✂ Close：先做一次膨胀操作，再做一次腐蚀操作，一般用于去除**黑色噪声**的处理。

空域变换

⌚ 腐蚀 (Erode)

⌚ 操作原理：在图像 (x,y) 像素点的邻域中，选择像素最小值作为像素点 (x,y) 的像素值。也就是**最小值滤波/卷积**。 ----> **池化**



空域变换

⌚ 膨胀 (Dilate)

⌚ 操作原理：在图像 (x,y) 像素点的邻域中，选择像素最大值作为像素点 (x,y) 的像素值。也就是**最大值滤波/卷积**。---> **最大池化**



空域变换

⌚ Open: (Erode -> Dilate)



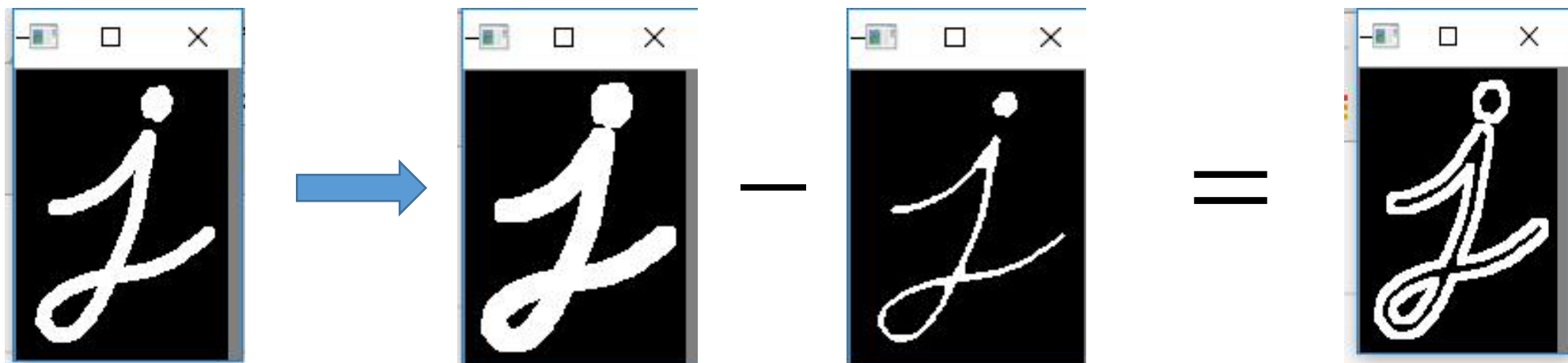
⌚ Close: (Dilate -> Erode)



空域变换

⌚ Morphological Gradient(形态梯度)

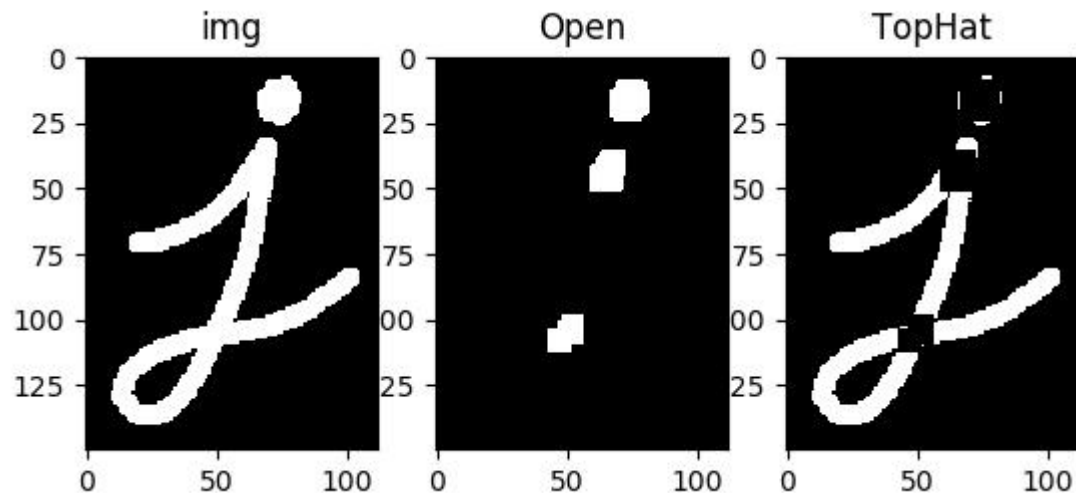
⌚ 在膨胀和腐蚀图像之间获取一个差集，也就是 $\text{Gradient} = \text{Dilate} - \text{Erode}$ ；该操作的作用能够提取边缘特征信息。



空域变换

⌚ Top Hat

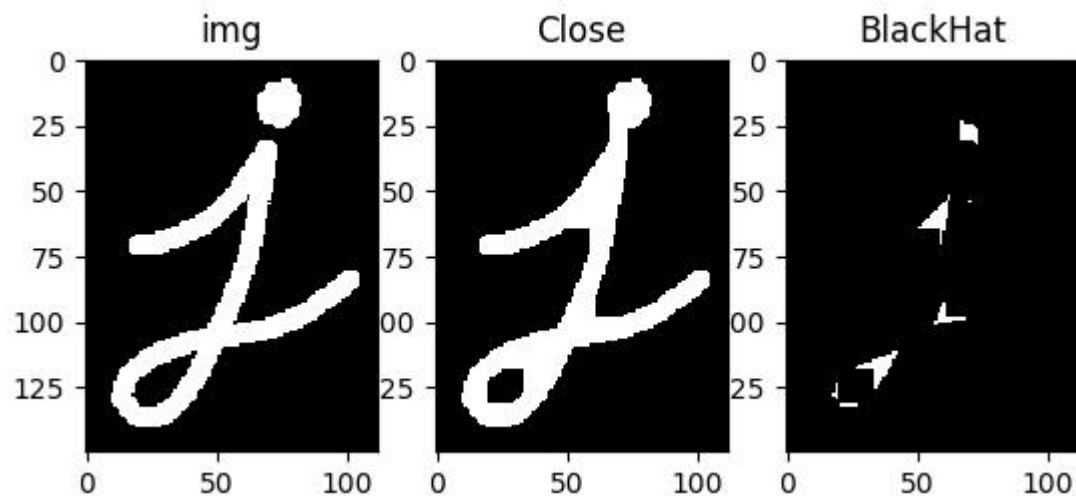
- ⌚ 在原始图像和Open操作的图像上做一个差集，也就是 $\text{Top Hat} = \text{image} - \text{Open}$ ；该操作的主要作用是可以提取一些非交叉点的信息。一般不用。



空域变换

⌚ Black Hat

- ⌚ 在Close操作的图像和原始图像上做一个差集，也就是 $\text{Black Hat} = \text{Close} - \text{image}$ ；该操作的主要作用是可以提取一些交叉点附件的位置特征信息。一般不用。



空域变换

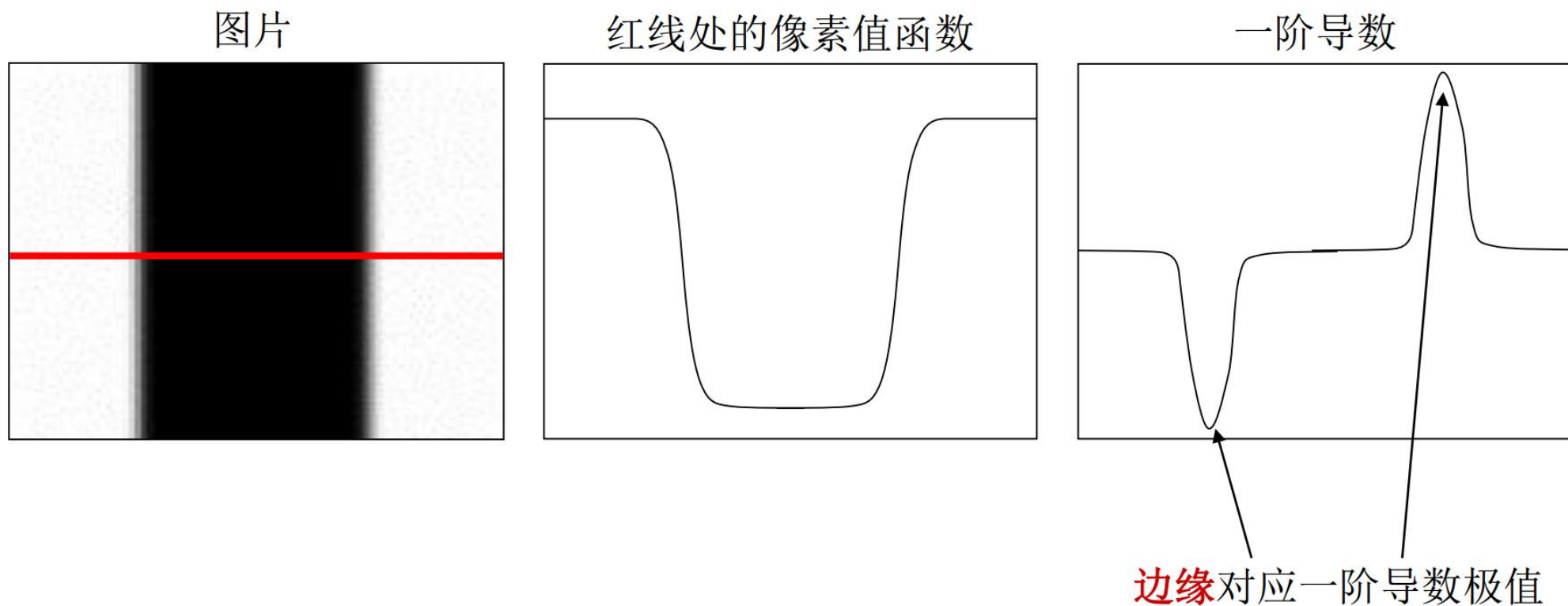
⌚ 除了前面介绍的普通滤波/卷积操作外，在图像空域上而言，还存在一些比较重要的特征信息，比如：边缘(Edge)特征信息；**边缘信息指的就是像素值明显变化的区域，具有非常丰富的语义信息，常用于物体识别**等领域。



空域变换

⌚ 边缘定义

⌚ 像素值**快速变化**的区域 --> 也就是像素值函数**一阶导数**的**极值区域**

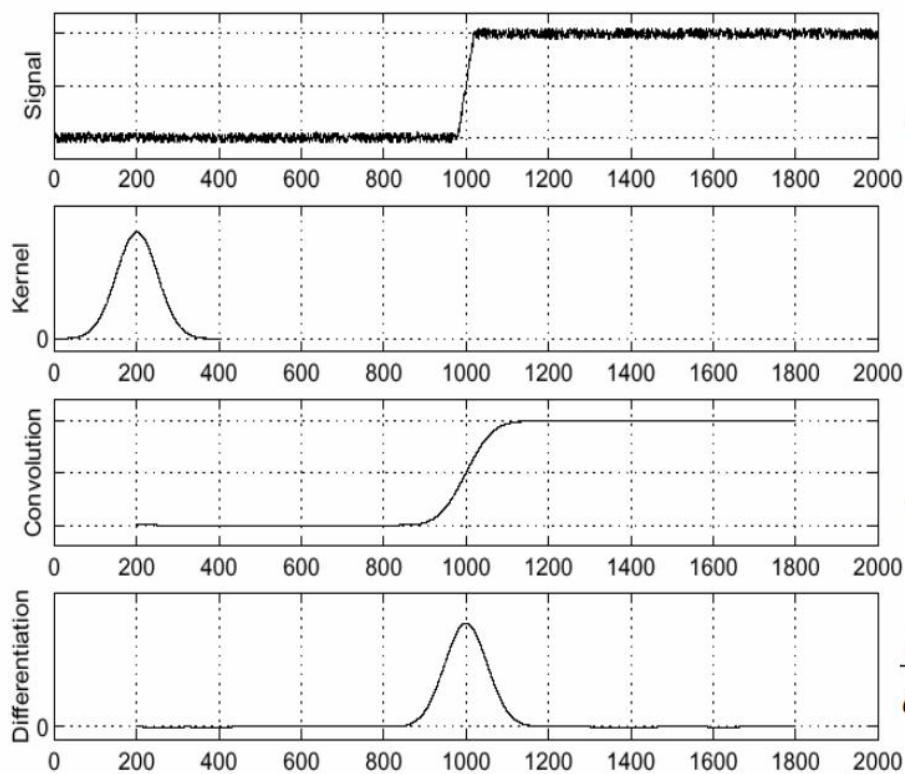


空域变换

⌚ 边缘提取

⌚ 先**高斯去噪**，再使用**一阶导数**获取极值 --> **Sobel滤波**

⌚ 导数对于噪声是敏感的。



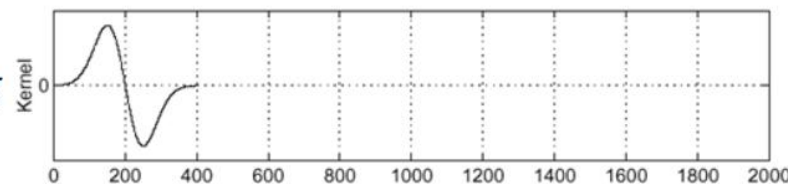
f

g

$f * g$

$\frac{d}{dx}(f * g)$

$$\frac{d}{dx}(f * g) = f * \frac{d}{dx}g$$



$$f * \frac{d}{dx}g$$

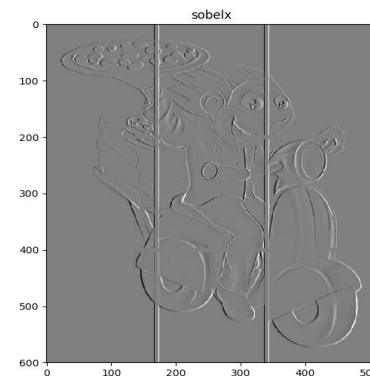
空域变换

⌚ 梯度Sobel滤波/卷积

⌚ 主要就是梯度和高斯平滑的结合，在求解梯度之前，首先进行一个高斯平滑的操作。

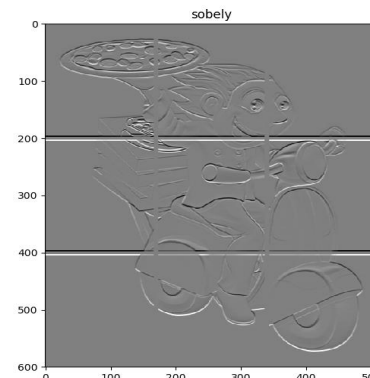
⌚ SobelX: 水平梯度/垂直边缘，提取垂直的边缘信息；

$$\begin{array}{c} \text{水平梯度} \rightarrow \\ \text{高斯平滑} \downarrow \end{array} \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 1 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{array}{c} \text{高斯平滑} \\ \text{水平梯度} \end{array}$$



⌚ SobelY: 垂直梯度/水平边缘，提取水平的边缘信息；

$$\begin{array}{c} \text{高斯平滑} \rightarrow \\ \text{垂直梯度} \downarrow \end{array} \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 1 \end{bmatrix} \otimes \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} \begin{array}{c} \text{垂直梯度} \\ \text{高斯平滑} \end{array}$$



空域变换

⌚ 梯度Laplacian滤波/卷积

⌚ 二阶微分算子作为梯度值

$$\Delta src = \frac{\partial^2 src}{\partial x^2} + \frac{\partial^2 src}{\partial y^2}$$

⌚ 效果:

⌚ 块团检测: 周边高于或者低于中心点的区域

⌚ 边缘检测: 像素值快速变化的区域

0	1	0
1	-4	1
0	1	0

1	1	1
1	-8	1
1	1	1

0	0	1	0	0
0	1	2	1	0
1	2	-16	2	1
0	1	2	1	0
0	0	1	0	0

空域变换

⌚ 梯度滤波/卷积

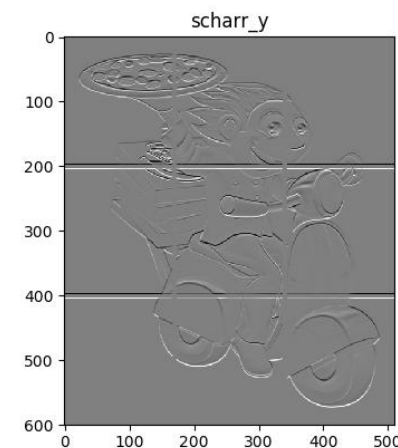
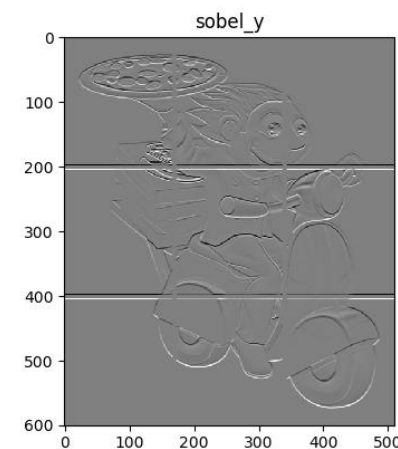
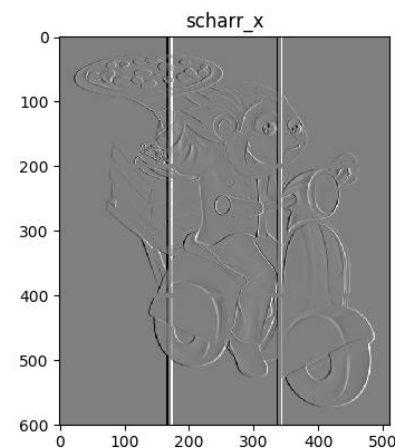
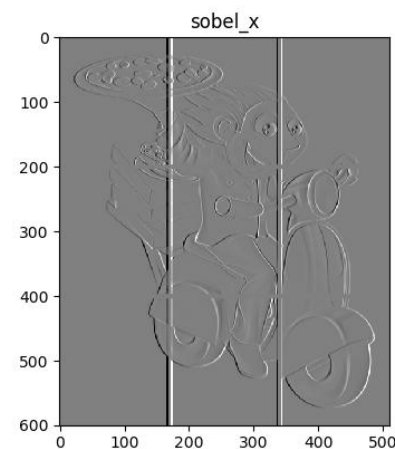
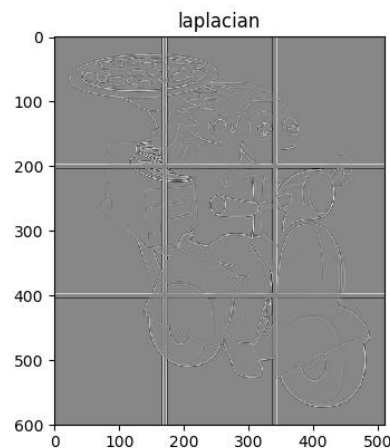
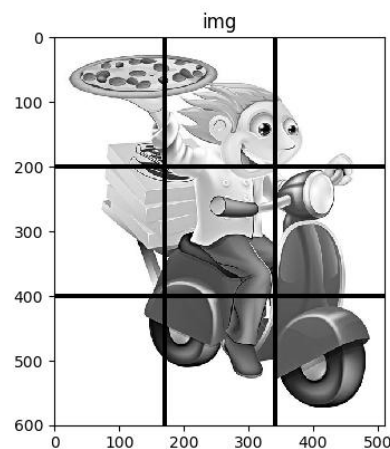
⌚ 主要功能就是提取边缘特征；

⌚ 主要实现方式：

⌚ Laplacian滤波

⌚ Sobel-X滤波

⌚ Sobel-Y滤波



空域变换

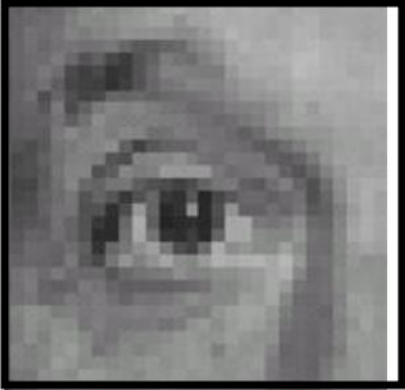
⌚ 其它滤波/卷积:

⌚ 左移

⌚ 锐化

 $\otimes$

0	0	0
0	0	1
0	0	0

 $=$ 

 $\otimes$

0	0	0
0	2	0
0	0	0

 $- \frac{1}{9}$

1	1	1
1	1	1
1	1	1

 $=$ 

Canny边缘检测算法

⌚ Canny边缘检测算法是从不同视觉对象中提取有用的结构信息，是传统计算机视觉中非常广泛应用的一种边缘提取算法。其Canny算法的主要分为5个步骤：

- ⌚ 使用高斯滤波器，以平滑图像，滤除噪声；
- ⌚ 计算图像中每个像素点的梯度大小和方向；
- ⌚ 应用非极大值(Non-maximum Suppression)抑制，以消除非边缘像素；
- ⌚ 应用双阈值(Double-Threshold)检测来确定真实和潜在的边缘；
- ⌚ 通过抑制孤立的弱边缘最终完成边缘检测。

Canny边缘检测算法

⌚ 梯度强度及方向：其梯度强度及方向使用Sobel算子进行计算。

$$G = \sqrt{G_x^2 + G_y^2}$$
$$\theta = \arctan(G_y / G_x)$$
$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad S_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

$$G_x = S_x * A = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} * \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} = \text{sum} \left(\begin{bmatrix} -a & 0 & c \\ -2d & 0 & 2f \\ -g & 0 & i \end{bmatrix} \right)$$

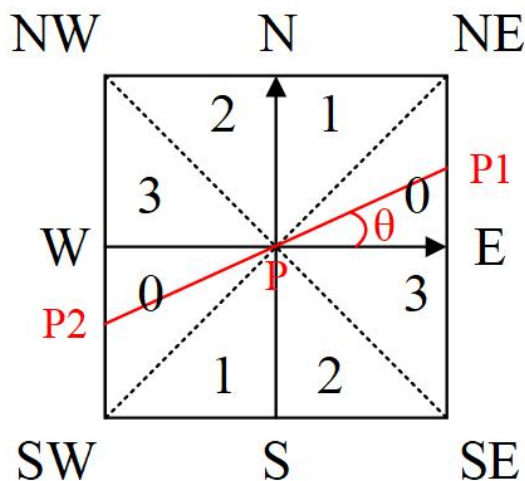
$$G_y = S_y * A = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} * \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} = \text{sum} \left(\begin{bmatrix} a & 2b & c \\ 0 & 0 & 0 \\ -g & -2h & -i \end{bmatrix} \right)$$

Canny边缘检测算法

⌚ 非极大值抑制：属于一种边缘稀疏技术，其主要作用就是**构建“瘦”边**。

非极大值抑制可以将局部最大值之外的所有梯度值抑制为0。其构建过程如下：

- ⌚ 将当前像素的梯度强度与沿着正负梯度方向上的两个像素进行比较；
- ⌚ 如果当前像素的梯度强度与另外两个像素相比最大，则该像素点保留为边缘点，否则该像素点被抑制。



Canny边缘检测算法

- ⌚ 双阈值检测：在进行非极大值抑制后，剩余的像素可以更准确地表示图像中的实际边缘，但是有可能还是存在一些噪音数据形成的边缘像素，所以需要使用弱梯度值来过滤这些非实际的边缘像素，并保留高梯度的实际边缘值。
即如果像素边缘的梯度值高于高阈值，则将其标记为强边缘像素，如果像素边缘的梯度值小于高阈值并且大于低阈值，则将其标记为弱边缘像素，如果像素边缘的梯度值小于低阈值，则会将其抑制。

Canny边缘检测算法

if $G_p \geq HighThreshold$

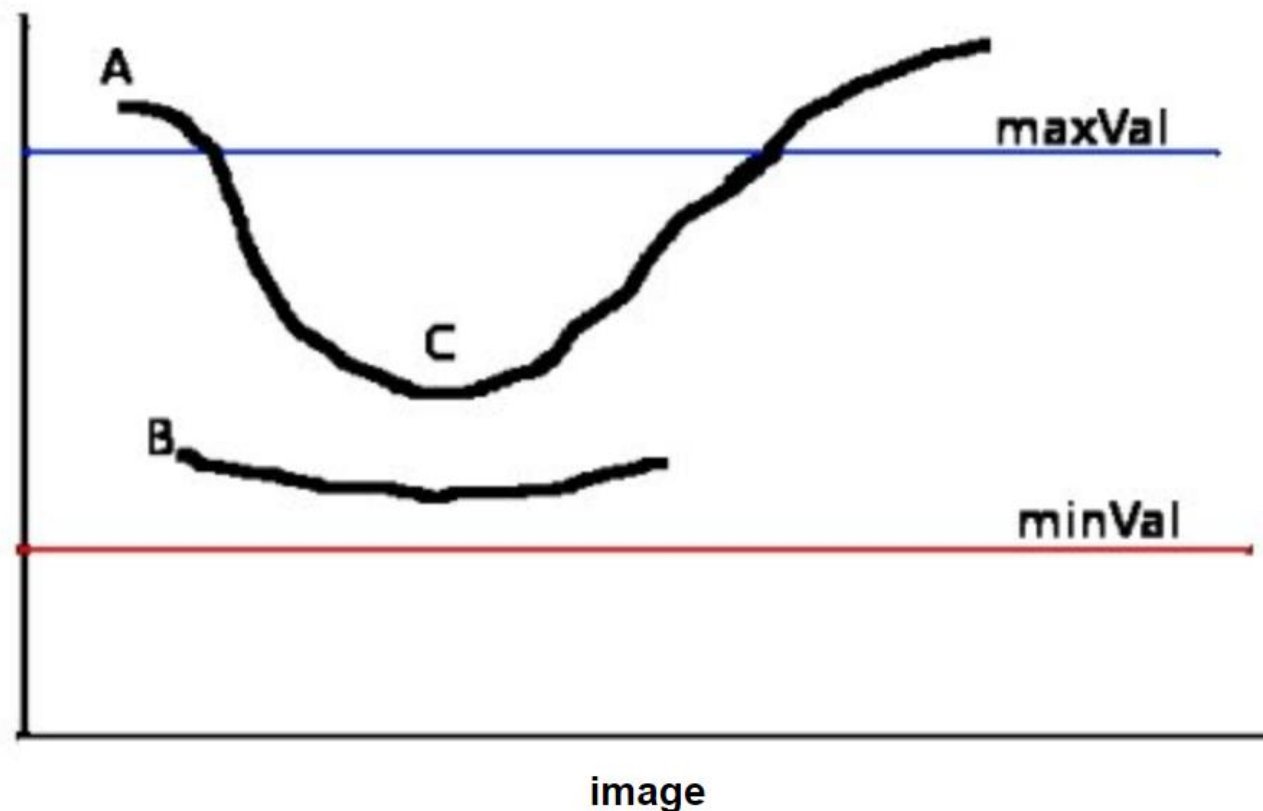
G_p is an strong edge

else if $G_p \geq LowThreshold$

G_p is an weak edge

else

G_p should be suppressed

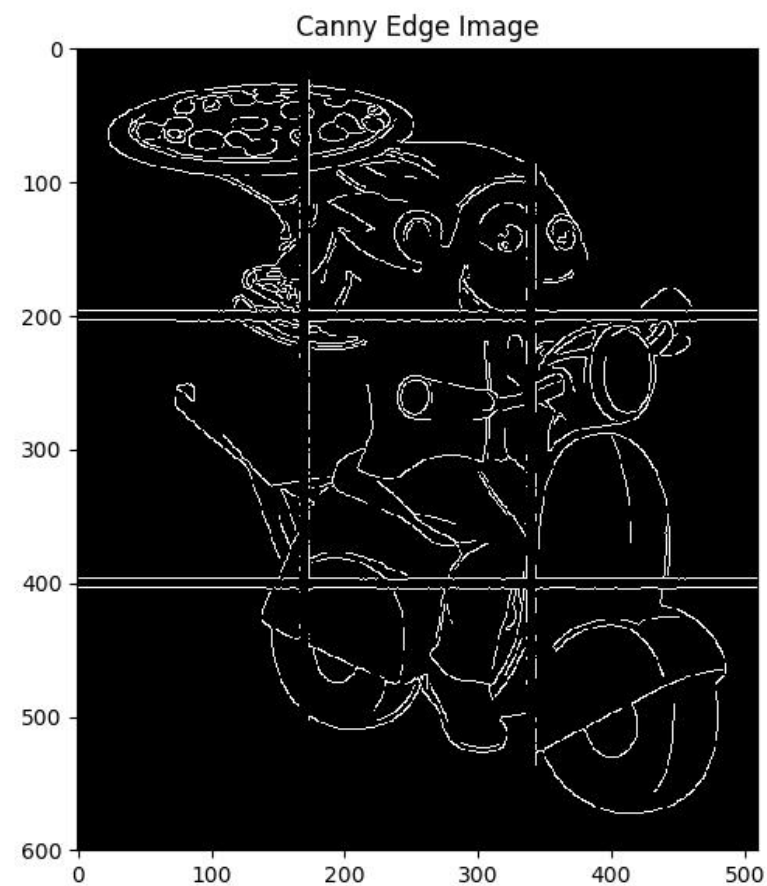
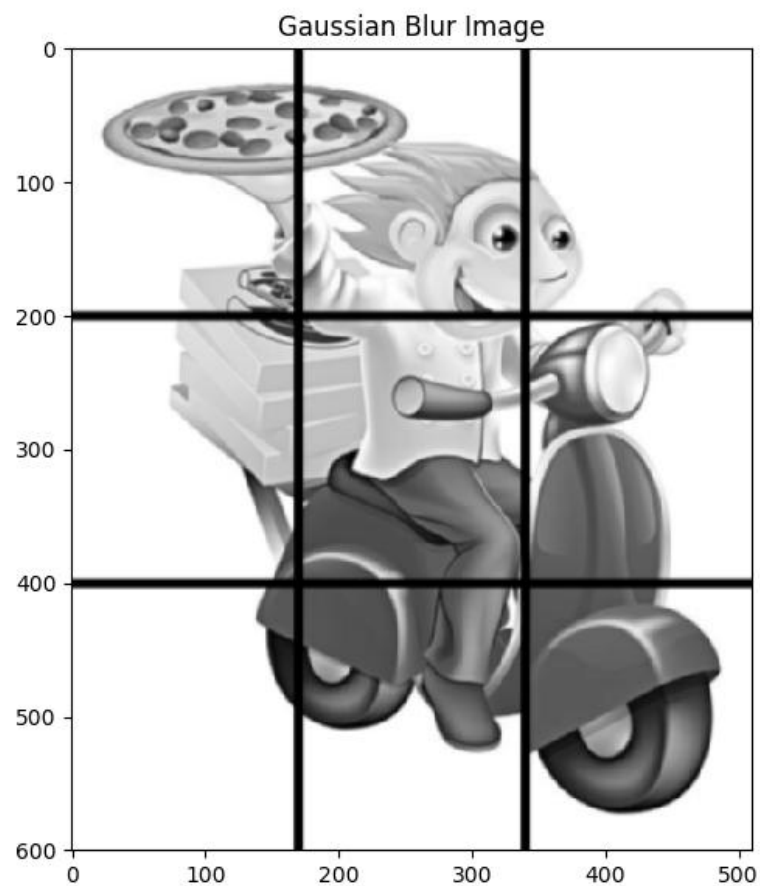
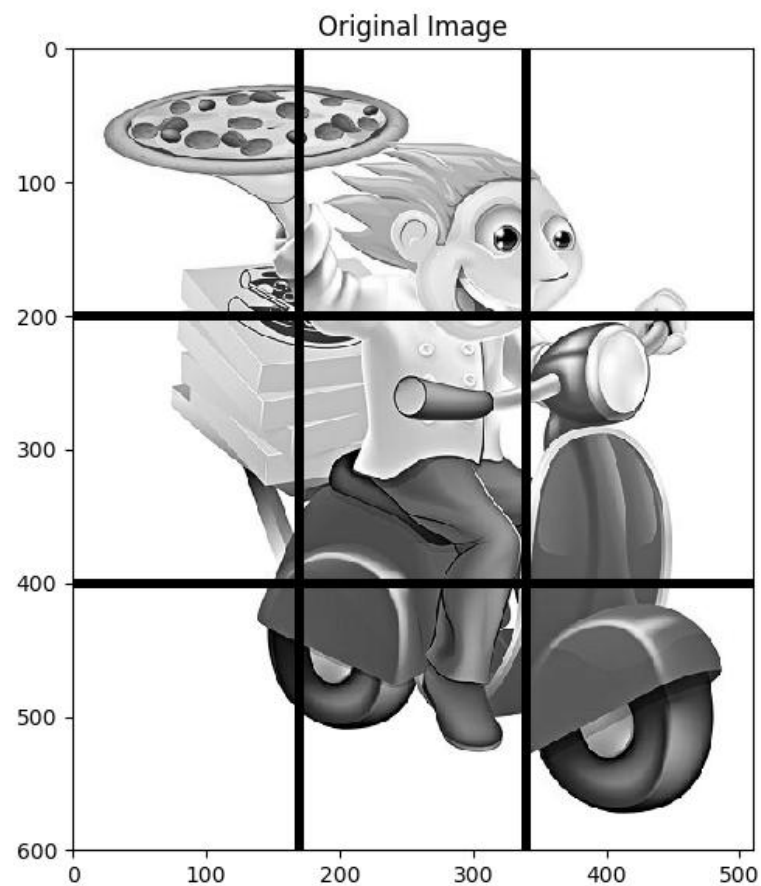


Canny边缘检测算法

- ⌚ 抑制孤立弱边缘：通过查看弱边缘像素及其周边的八个邻域像素，主要其中有一个为强边缘像素/真实边缘像素，则该弱边缘点被保留为真实边缘/强边缘像素。其算法原理是：真实的弱边缘像素一定是连接强边缘像素点的。

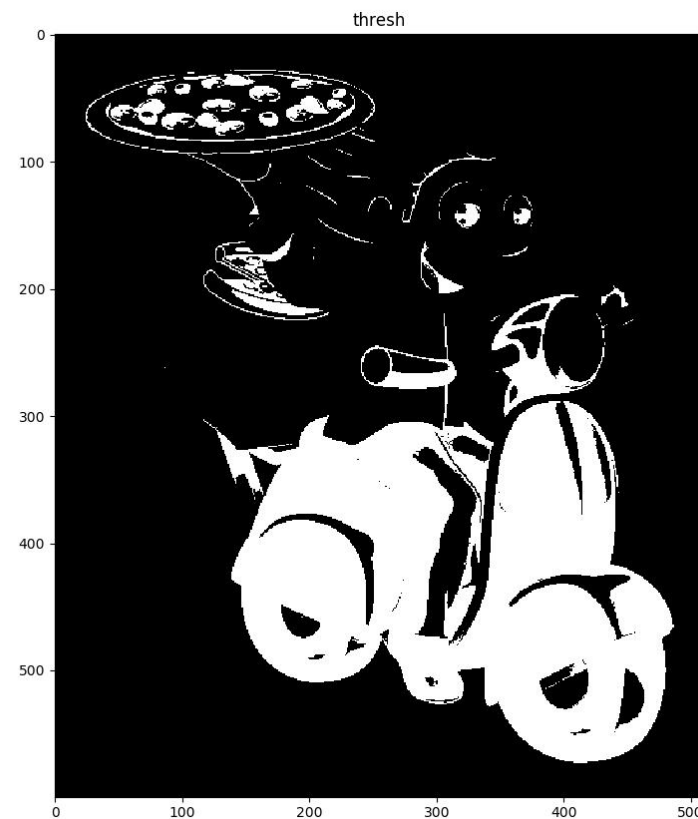
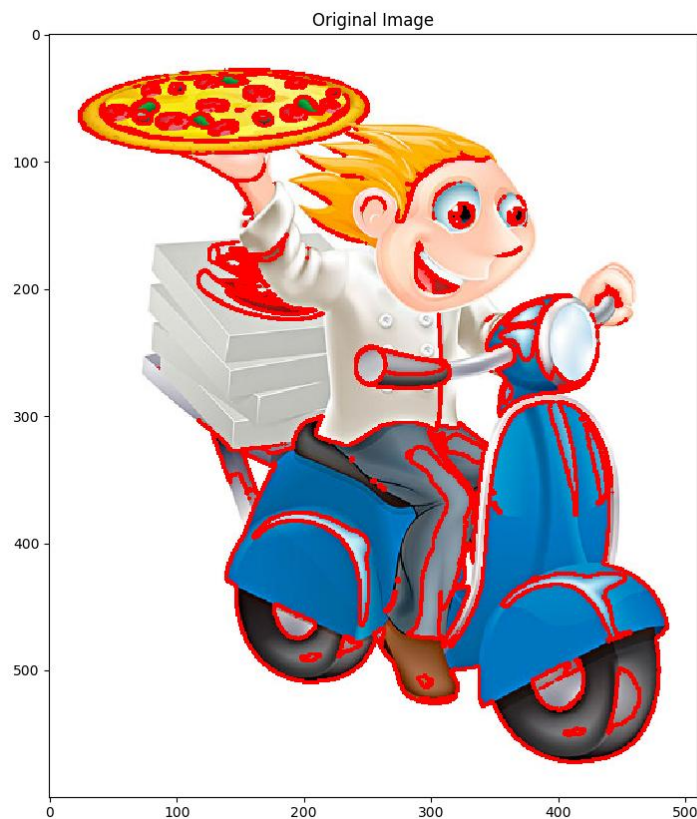
if $G_p == LowThreshold$ and G_p connected to a strong edge pixel
 G_p is an strong edge
else
 G_p should be sup pressed

Canny边缘检测算法



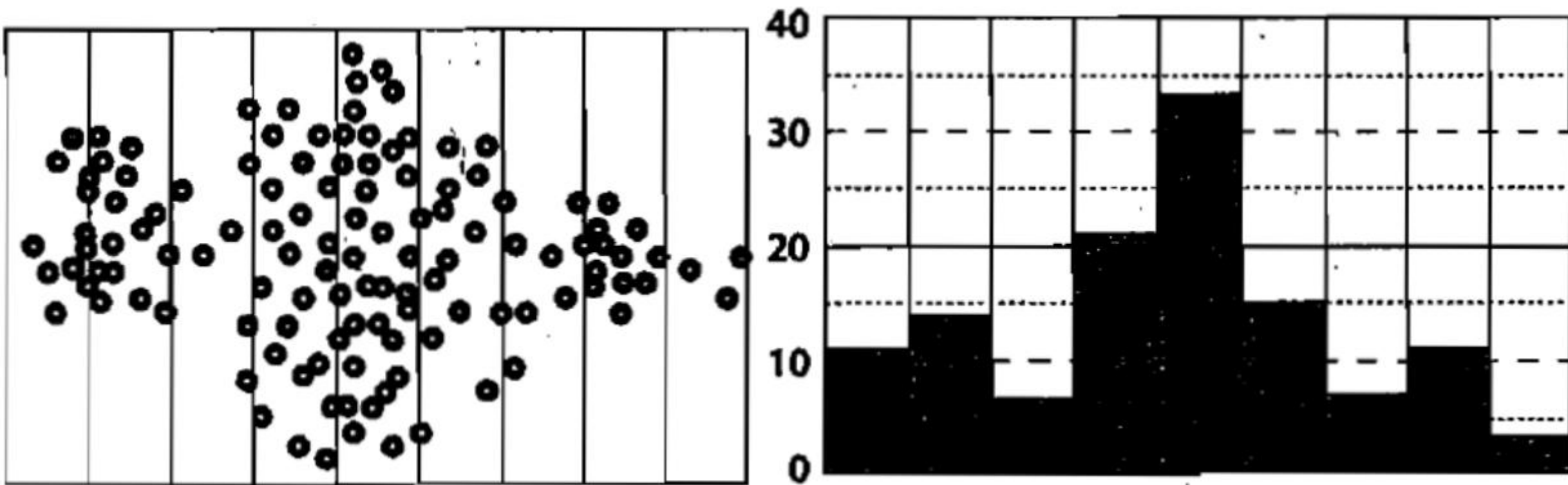
轮廓信息

⌚ 轮廓信息可以简单的理解为从图像中获取图像曲线的连接点信息，在目标检测、目标识别等业务场景中，具有一定的效果；简单来讲，轮廓信息就是从图像中获取图像边缘信息。

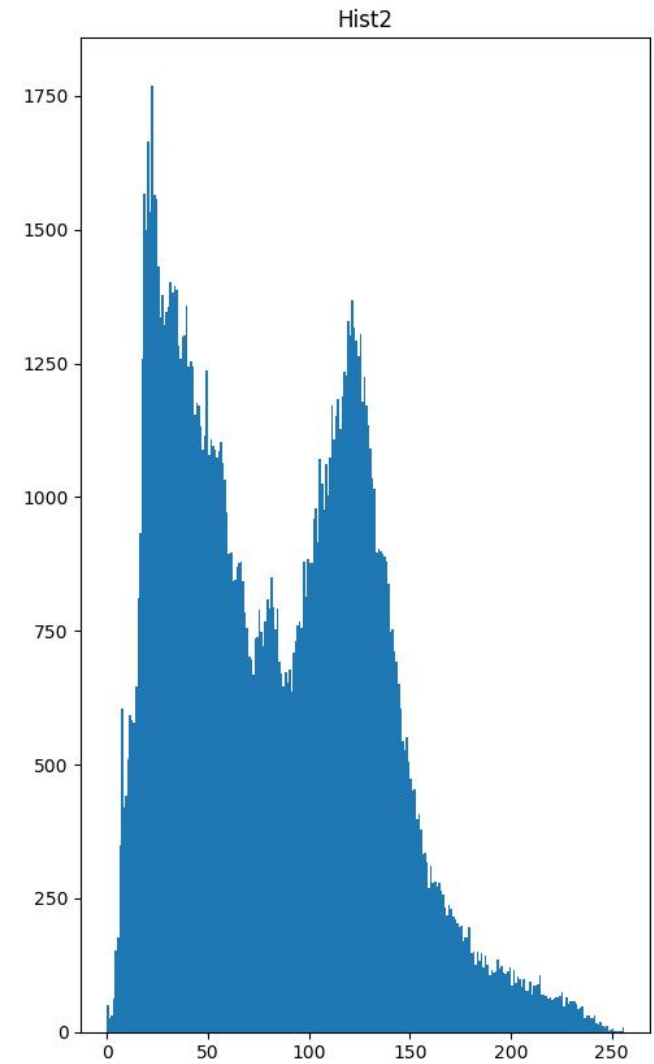
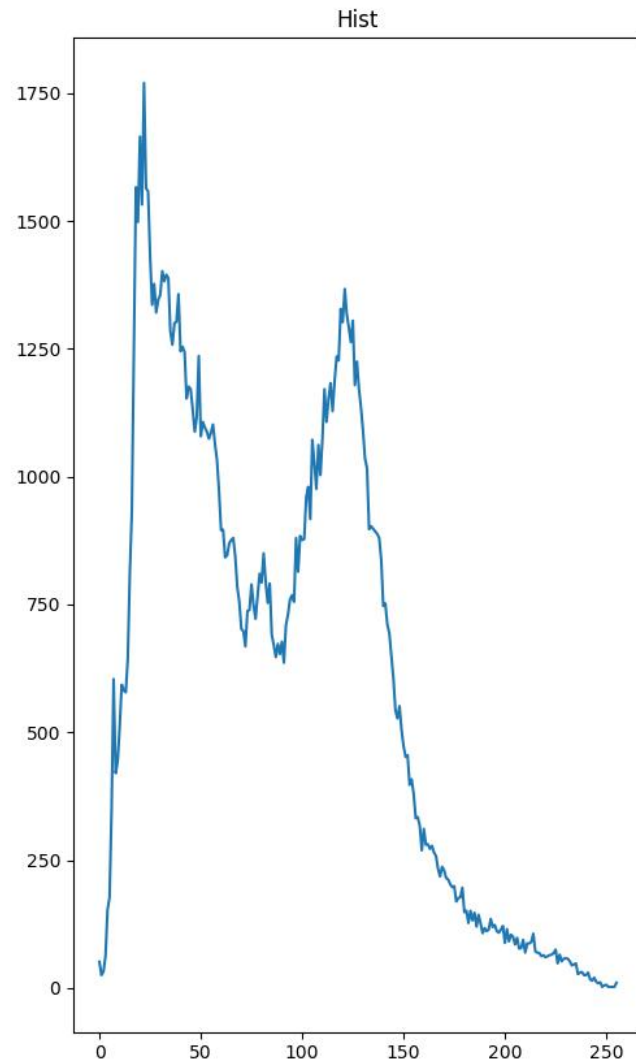
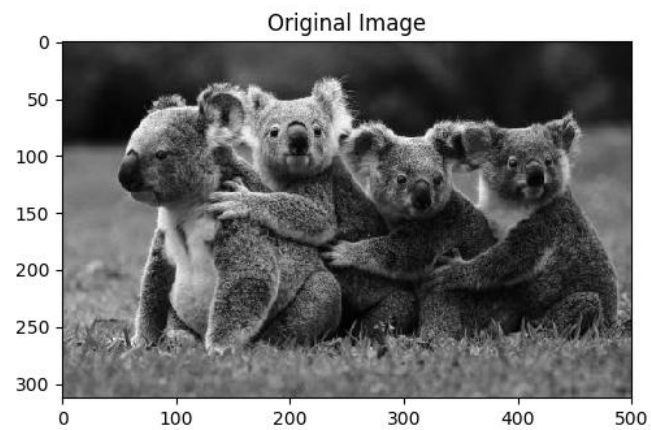


直方图

🕒 直方图(Histogram)是一种可用于计算图片特征(Feature)的常规方式。也就是对图片数据的特征分布的一种统计。其统计方式一般采用人工统计或者聚类统计。

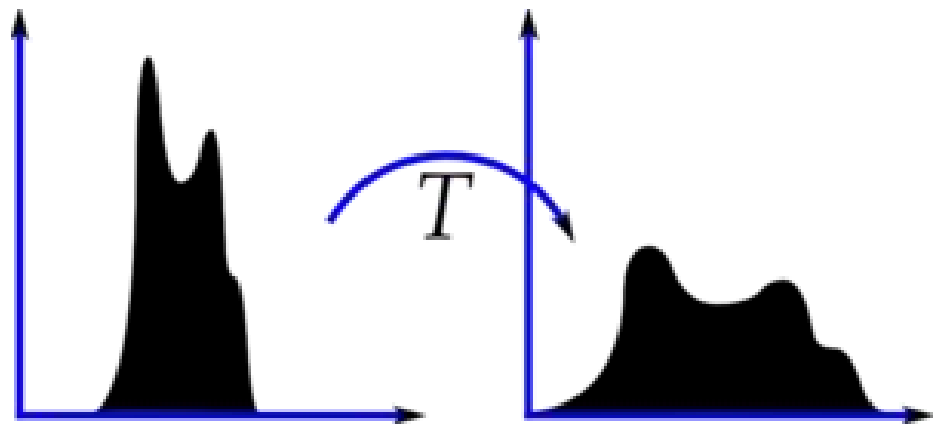


直方图

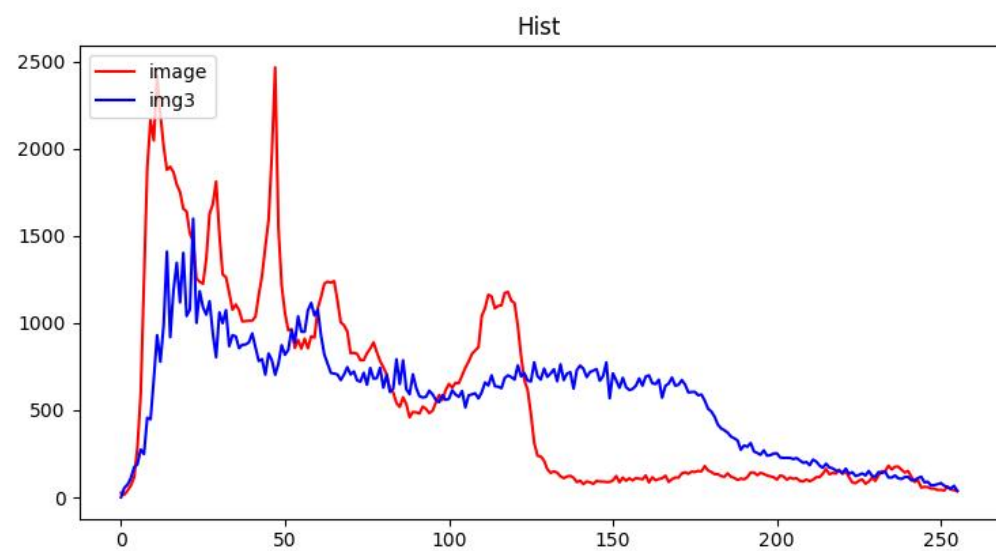
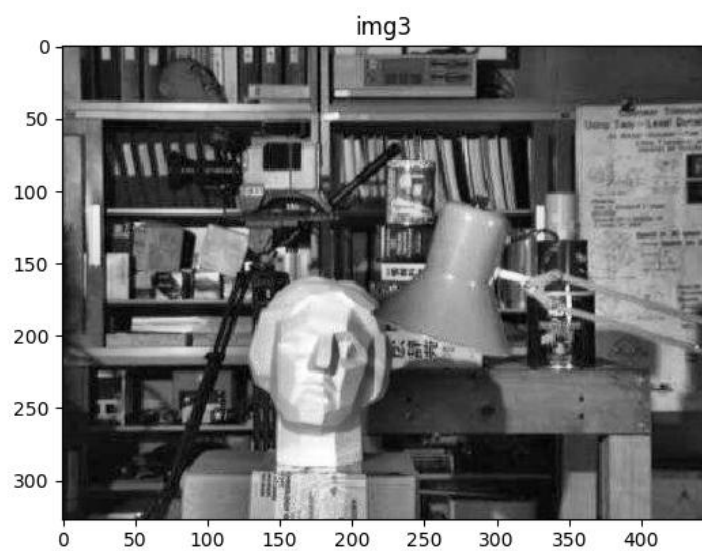
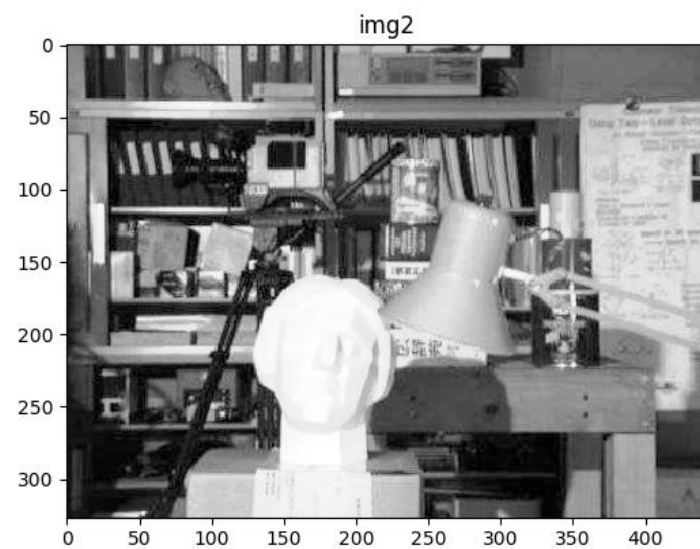
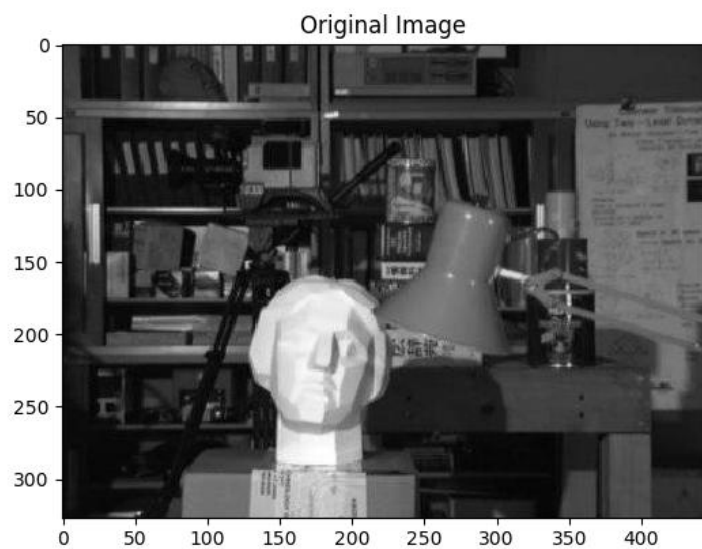


直方图

⌚ 如果某一个图像的像素仅限制于某个特定的小范围，那么图像看上去就会不太正常，因为实际上一个比较好的图像像素点应该是在某一个范围内，所以需要将该像素的直方图做一个拉伸的操作；比如：一个偏暗的图像，像素基本上都在较小的位置，如果将像素值增大，不就可以让图像变亮嘛！实际上，**直方图均衡化**操作是一个提高图像对比度的方式。



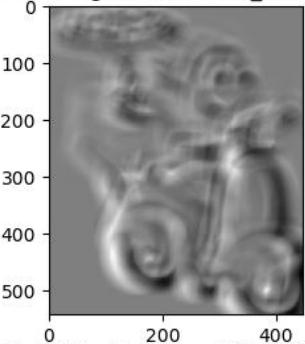
直方图



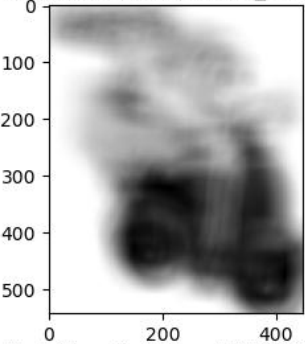
模板匹配



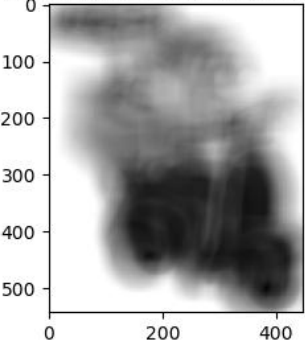
Matching Result cv.TM_CCORR



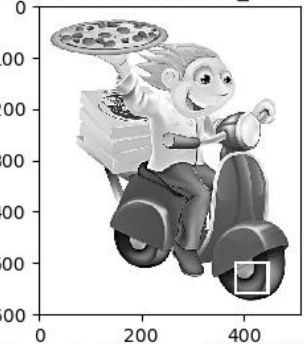
Matching Result cv.TM_CCORR



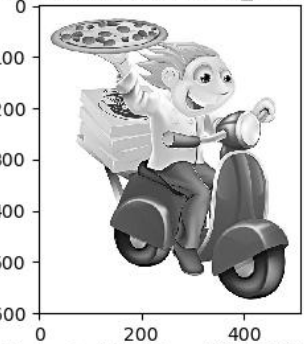
Matching Result cv.TM_SQDIFF



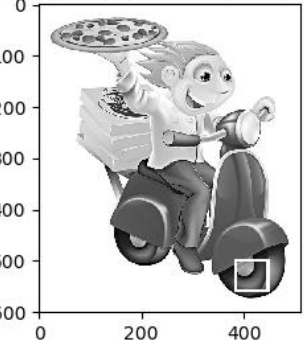
Detected Point cv.TM_CCORR



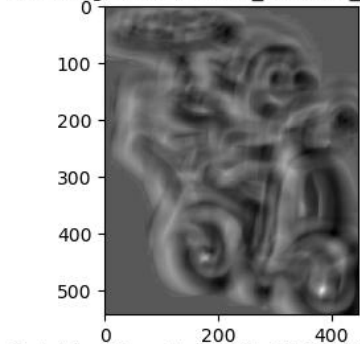
Detected Point cv.TM_CCORR



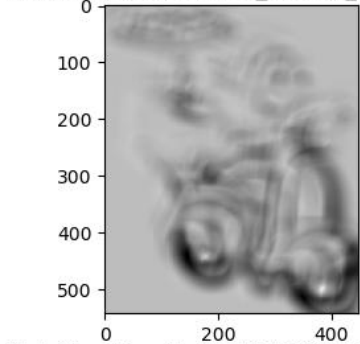
Detected Point cv.TM_SQDIFF



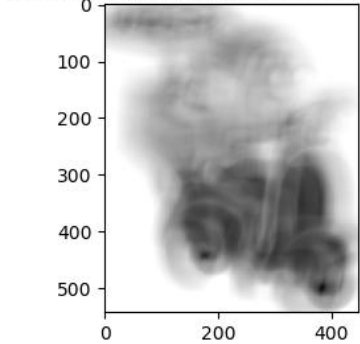
Matching Result cv.TM_CCORR_NORMED



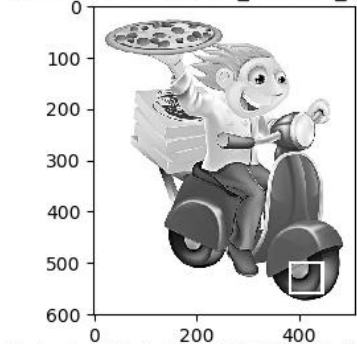
Matching Result cv.TM_CCORR_NORMED



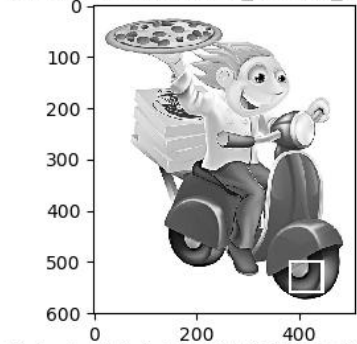
Matching Result cv.TM_SQDIFF_NORMED



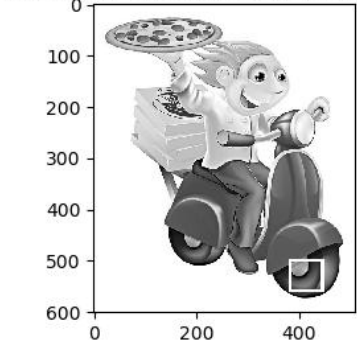
Detected Point cv.TM_CCORR_NORMED



Detected Point cv.TM_CCORR_NORMED



Detected Point cv.TM_SQDIFF_NORMED



图像特征提取_HOG特征

- ⌚ HOG特征(方向梯度直方图, Histogram of Oriented Gradient), 是一种在计算机视觉和图像处理中用来进行物体检测的特征描述子, 通过计算和统计图像局部区域的梯度方向直方图来构造特征, HOG特征结合SVM分类器被广泛应用与行人检测中。
- ⌚ 主要思想: 局部目标的表象和形状可以被梯度或者边缘的方向密度分布进行很好的描述, 本质上是进行梯度的统计信息, 而梯度主要存在于边缘位置。

图像特征提取_HOG特征

⌘ HOG特征实现过程:

- ⌘ 图像灰度化
- ⌘ 采用Gamma校正法对灰度图像进行颜色空间的标准化/归一化, 目的是为了调节图像的对比度, 降低图像局部的阴影和光照变化所造成的影响, 同时可以抑制噪音的干扰。
- ⌘ 计算图像每个像素的梯度(大小&方向), 目的是为了提取轮廓信息。
- ⌘ 将图像划分为小的cells(eg, 6*6像素区域)
- ⌘ 统计每个cell的梯度直方图(不同梯度的个数), 形成每个cell的descriptor;
- ⌘ 将多个cell组成一个block(eg: 3*3个cell形成一个block), 一个block中所有cell的特征descriptor串联起来得到当前block的HOG特征(会将一个block块中的所有特征向量做一个归一化处理, 可以降低对于光照的敏感性, 这里的归一化采用的是L2范数归一化);
- ⌘ 将图像image内的所有block的HOG特征串联起来得到当前图像的HOG特征, 也就是分类器对应的特征向量。

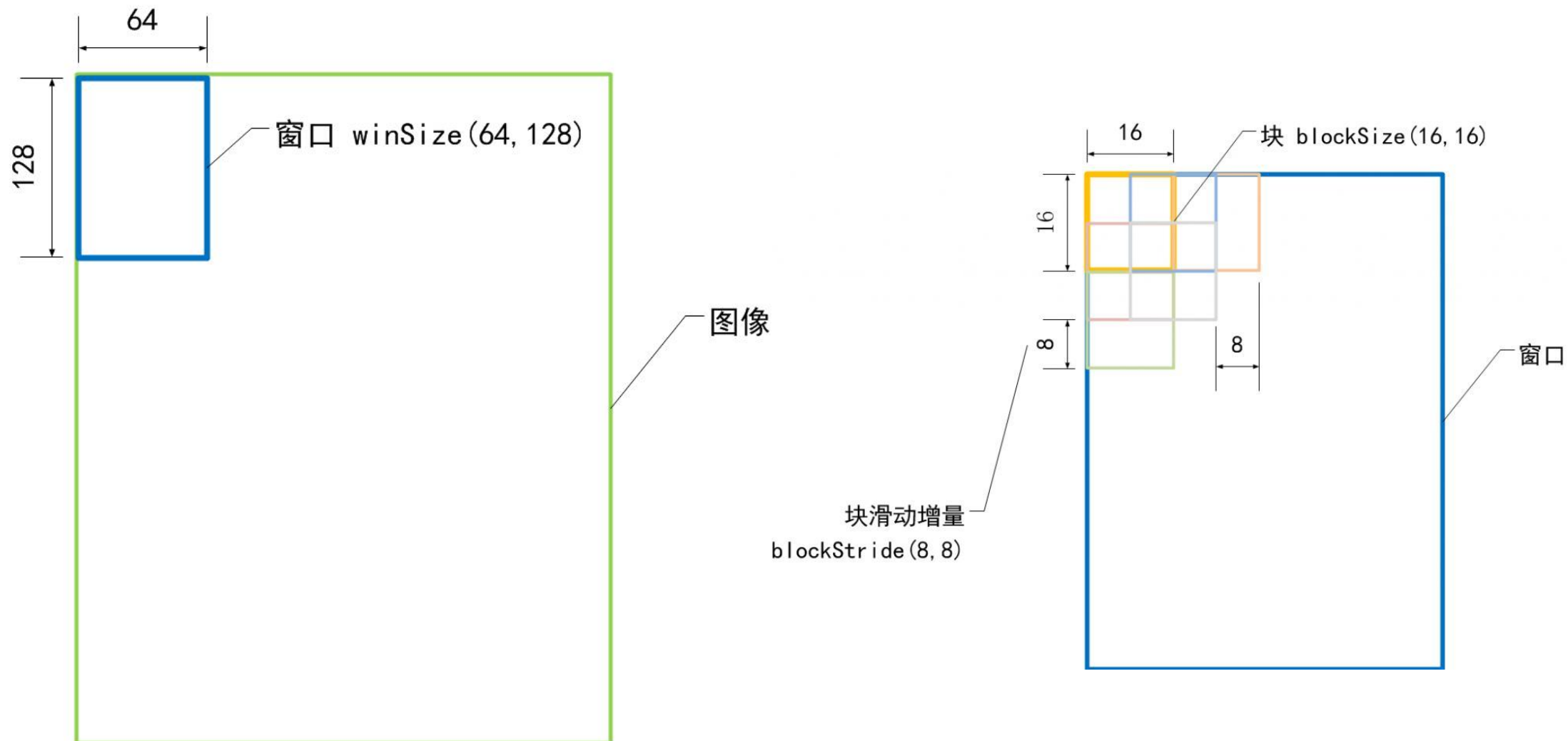
图像特征提取_HOG特征

- ⌚ 将样本图像分割为若干个像素单元(cell)，把梯度方向平均划分为9个区间(bin)，在每个单元中对所有像素的梯度方向在各个方向区间进行直方图统计，得到一个9维的特征向量；每相邻的4个cell组成一个block，把一个block中的特征向量串联形成36维的特征向量；用block对窗口进行扫描，步长为1像素，最终将所有块形成的特征串联起来，得到最终的特征向量。

图像特征提取_HOG特征

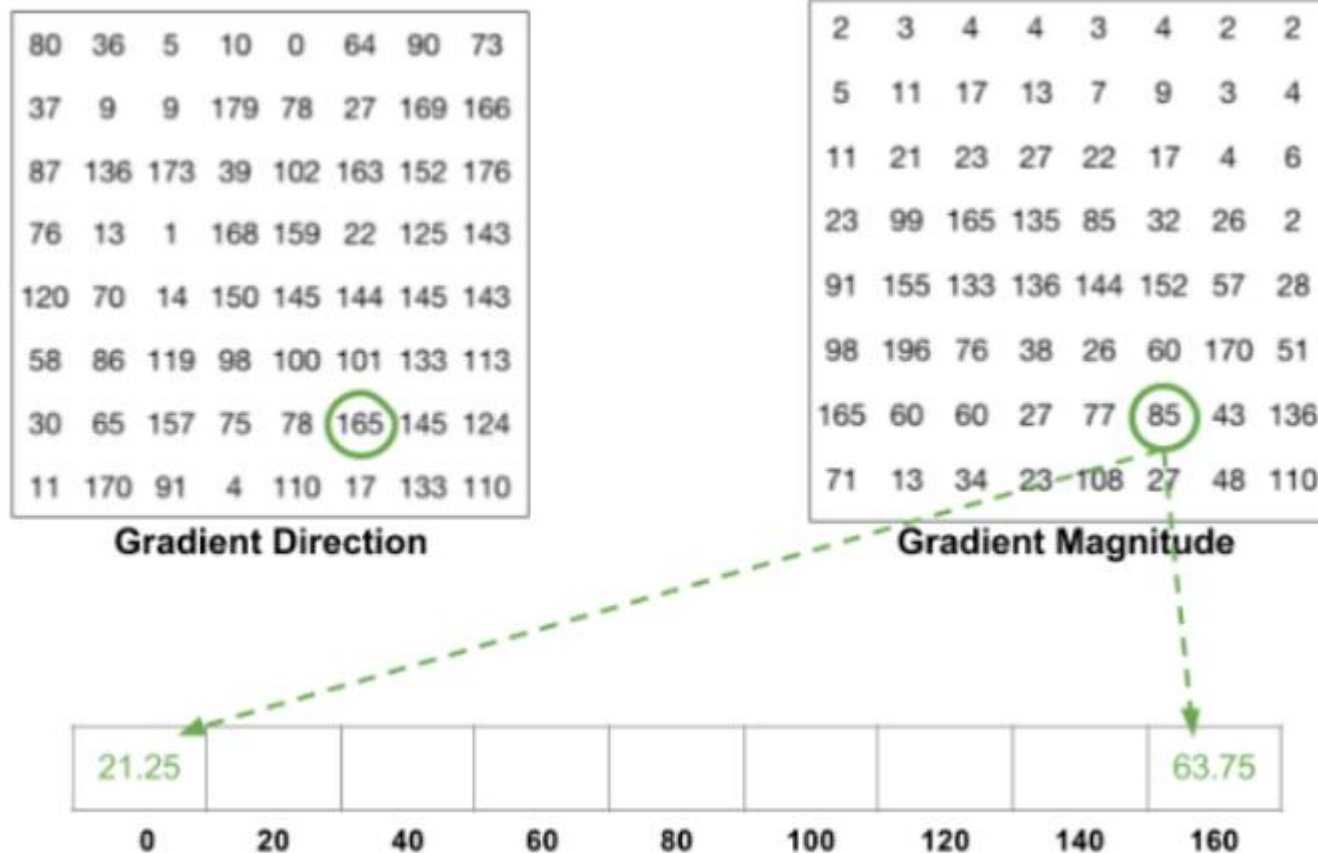
- ⌚ 假设 $64*128$ 的图像，每个cell大小为 $16*16$ ，每 $2*2$ 个cell形成一个block，每个cell中有9个特征，那么每个block中有 $4*9=36$ 个特征，以8个像素为步长，那么水平方向将有5个扫描窗口，垂直方向将有13个扫描窗口，也就是说 $64*128$ 的图像，最终HOG特征数目为： $36*5*13=2340$ 个特征。
- ⌚ 然后窗口会在图像上进行滑动。

图像特征提取_HOG特征



图像特征提取_HOG特征

⌚ 8 * 8大小的cell, 分成9个bins

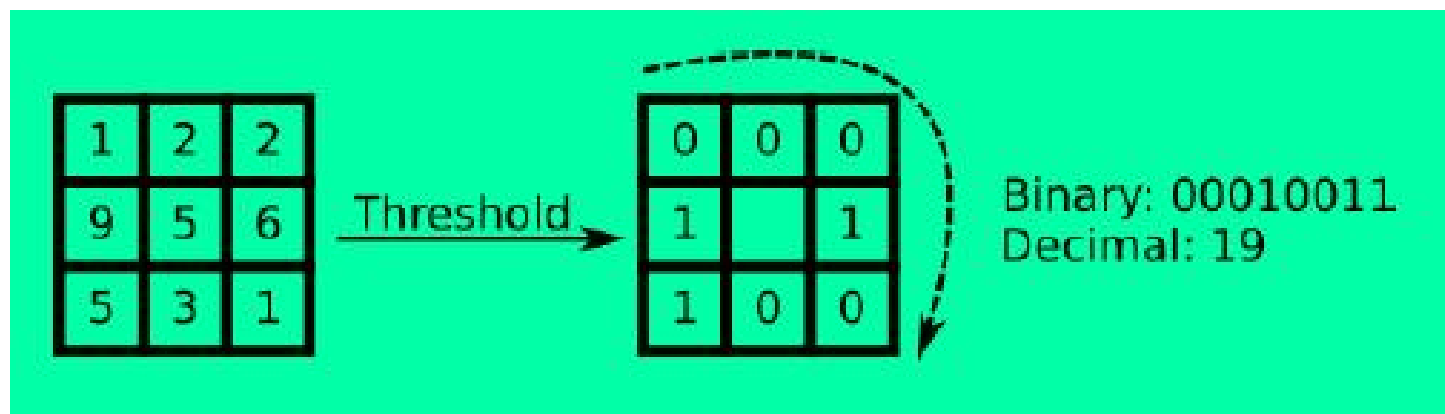


图像特征提取_LBP特征

- ⌚ LBP(Local Binary Pattern, 局部二值模式)是一种用来描述图像局部纹理特征的算子；具有旋转不变性和灰度不变性等优点，主要用于提取图像的局部纹理特征。

图像特征提取_LBP特征

⌚ 主要思想：在一个 3×3 的窗口中，以窗口中心像素为阈值，将相邻的8个像素的灰度值与其进行比较，若周围像素值大于中心像素值，则该像素点的位置被标记为1，否则为0，然后将这个 3×3 邻域中经过比较产生的8位二进制数转换为十进制，即可得到该窗口中心像素点的LBP的值，并使用这个值反应该区域的纹理特征信息。



图像特征提取_LBP特征

⌚ LBP特征实现过程:

- ⌚ 首先将检测窗口划分为 $16*16$ 的小区域(cell);
- ⌚ 对于每个cell中的一个像素, 将相邻的8个像素的灰度值与其进行比较, 若周围像素值大于中心像素值, 则该像素点的位置被标记为1, 否则为0。这样, $3*3$ 邻域内的8个点经比较可产生8位二进制数, 即得到该窗口中心像素点的LBP值;
- ⌚ 计算每个cell的直方图, 即每个数字出现的次数, 然后对该直方图做归一化处理(一般为L2范数归一化)。
- ⌚ 最终将所有cell的统计直方图的值串联起来形成一个特征向量, 即为整个图像的LBP纹理特征向量。

图像特征提取_LBP特征

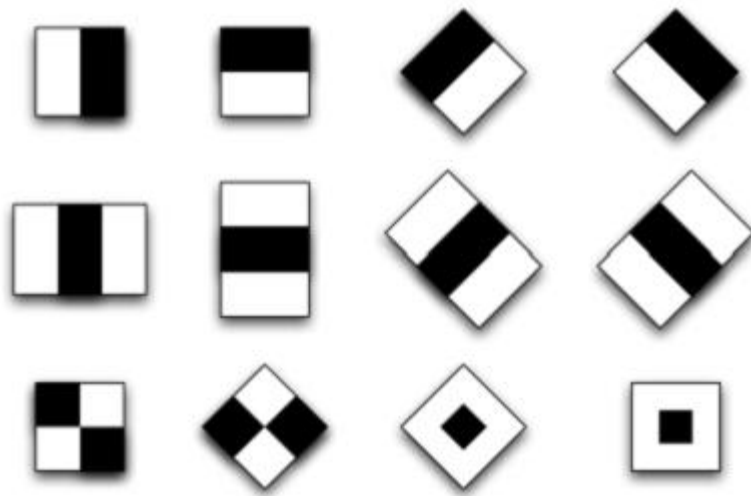
- ⌚ 将样本图像分割为若干个像素单元(cell)，计算每个像素的LBP值，把LBP值划分为9个区间(bin)，在每个cell中对LBP值进行分区统计，得到一个9维的特征向量；将图像的所有cell的特征串联起来，得到最终的特征向量。
- ⌚ 假设64*128的图像，每个cell大小为16*16，每个窗口大小为3*3，每个cell中有9个特征；cell与cell之间不重叠，那么水平方向将有4个cell，垂直方向将有8个cell，也就是说64*128的图像，最终LBP特征数目为：
 $9*4*8=288$ 个特征。

图像特征提取_Haar特征

⌚ Haar特征分为三类：**边缘特征**、**线性特征**、**中心特征**和**对角线特征**，组合成特征模板。特征模板内有白色和黑色两种矩形，并定义该模板的特征值为白色矩形像素和减去黑色矩形像素和。Haar特征值反映了图像的灰度变化情况。例如：脸部的一些特征能由矩形特征简单的描述，如：眼睛要比脸颊颜色要深，鼻梁两侧比鼻梁颜色要深，嘴巴比周围颜色要深等。但矩形特征只对一些简单的图形结构，如边缘、线段较敏感，所以只能描述特定走向（水平、垂直、对角）的结构。

人脸检测

⌚ Harr级联是一种基于Haar特征的级联分类器，类似于集成算法，即将多个弱分类器串联成一个强分类器的过程。其中Haar特征即边缘特征、线性特征、中心特征以及对角线特征，然后将这些特征进行组合合并，最终就可以用于某些物体的检测。



THANKS!